

Cloud 용어 개념 정의

- [I. Cloud 기본 용어 부문]
- [II. Cloud 서비스 유관 용어 부문]
- [III. Cloud 인프라기술/개발 관련 용어 부문]
- [IV. Cloud 서비스 보안 관련 용어 부문]

목 차

[I. Cloud 기본 용어 부문]

1. Cloud Computing	2
2. On-Premise	3
3. Bare Metal Model [I Type]	4
4. CSP vs CSC	5
5. MSP, ISV	6
6. Cloud Native	7
7. Micro-service Architecture	8
8. Tenant	9
9. Virtual Infra-structure	10
10. Hypervisor	11
11. Auto-scaling	12

[II. Cloud 서비스 유관 용어 부문]

1. Public / Private / Hybrid / Multi Cloud	13
2. IaaS / PaaS / SaaS	22

[III. Cloud 인프라기술/개발 관련 용어 부문]

1. SDx (Software Defined Anything)	30
2. NFV (Network Function Virtualization) .	31
3. Container, Docker	32
4. Kubernetes	33
5. DevOps	35
6. DevSecOps	37
7. Agile	38
8. Cloud to Edge	39
9. Cloud 관련 스토리지 개념 이해	41

[IV. Cloud 서비스 보안 관련 용어 부문]

1. SECaaS	42
2. CASB	43
3. CWPP	44
4. CSPM	45

[V. 별첨]

1. workload	46
2. IaC	47

Cloud Computing

□ 정의: 서버, 스토리지, 데이터베이스, 네트워크, 소프트웨어 등의 컴퓨팅 서비스를 공유 네트워크("Cloud")을 통해 제공하는 것

□ 목적: 비즈니스 규모에 맞춰 합리적인 비용으로 비즈니스 서비스를 효율적으로 운영할 수 있도록 환경 제공

□ Cloud 컴퓨팅은 하기 본문의 5가지 필수 특성을 가지고 있음

Cloud 컴퓨팅 주요 특징

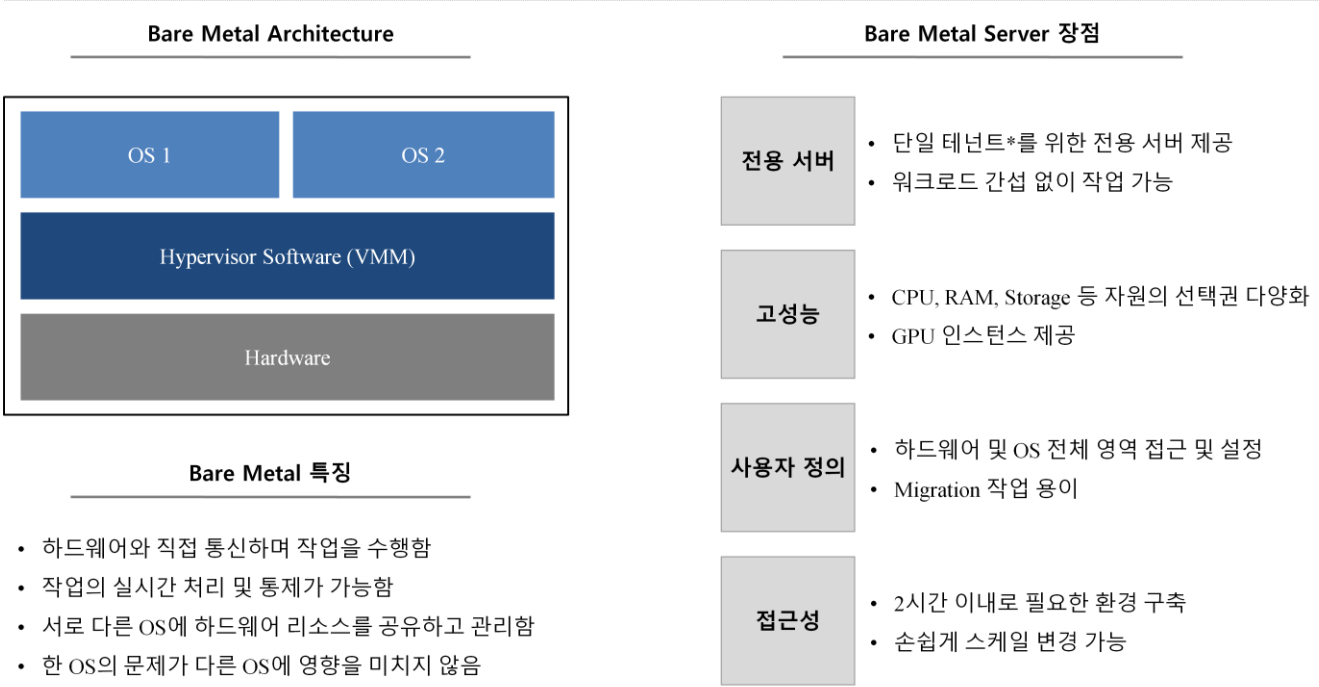
On-demand self-service • 주문형 셀프 서비스 - 고객은 제공사의 도움 없이 필요에 따라 자동적으로 서버 시간과 네트워크 저장소와 같은 컴퓨팅 기능을 단독으로 준비할 수 있음	Rapid elasticity • 빠른 탄력성 - 기능은 탄력적으로 준비되고 배포 가능함 - 자동으로 요구에 비례하여 빠르게 규모를 확장 또는 축소 가능함
Broad network access • 광범위 네트워크 접근 - 모든 기능들은 네트워크를 통해 이용 가능함 - 기기종 클라이언트 플랫폼(예: 모바일폰, 태블릿, 노트북, 워크스테이션)에서 접근 가능함	Measured service • 서비스 측정 - 서비스 유형(예: 저장소, 프로세싱, 대역폭, 활성계정)에 맞춰 측정 기능을 활용하여 리소스를 자동으로 제어하고 최적화함 - 자원 사용은 감시, 제어, 보고되어 서비스 이용자와 제공자 모두에게 투명성을 제공함
Resource pooling • 리소스 풀링 - 고객 요구사항에 따라 물리/가상화된 자원들을 멀티 테넌트 모델을 사용하는 복수의 고객들에게 제공하기 위해 공유 관리함 - 고객이 제공받는 자원의 정확한 위치에 대한 정보 없이 장소 독립적으로 운용 가능함	

※ 출처: The NIST Definition of Cloud Computing(NIST, 2011)

On-Premise	□ 정의: 소프트웨어 등을 (원격) 공유 환경이 아닌 자체적으로 보유한 HW 자산에 직접 설치해 운영하는 방식을 의미함 □ Cloud 컴퓨팅에 반대되는 개념으로 전통적인 기업 인프라 구축 방식을 의미함	
구분	On-Premise	Cloud Platform(SaaS)
호스팅	- 기업 자체 서버를 통해 솔루션이 제공됨 - 사내 네트워크 등 접근에 대한 제한이 있음	- 서비스 공급 업체 서버를 통해 솔루션 제공됨 - 인터넷에 연결된 모든 장치에서 접근 가능함
비용	하드웨어, 솔루션 등 도입 비용이 많이 필요함	월간 또는 연간으로 솔루션 사용 비용을 지불함
구축 시간	인프라, 솔루션 설치 등 구축 시간이 오래 걸림	준비된 환경을 배포만하기 때문에 구축 시간이 짧음
사용자 정의	기업 환경에 최적화된 솔루션 사용이 용이함	사용자 요구에 맞는 맞춤형 솔루션 설치가 어려움
유지보수	버그, 최신패치 등 유지보수 비용이 발생함	버그, 최신패치 등을 공급자가 유지보수를 관리함
스케일	스케일 변경 시 많은 비용과 시간이 필요함	자원 사용량을 즉각적으로 변경하여 쉽게 이용함

- Bare Metal
- 정의: Cloud 가상화 아키텍처에서는 VMM(Hypervisor 등)이 하드웨어와 직접 통신하는 방식의 기술을 의미함
舊 bare Metal의 정의: 일반적으로 어떤 소프트웨어도 담겨 있지 않은 하드웨어를 의미함 (* OS가 설치되지 않은 하드웨어)

□ 단일 테넌트를 위한 고성능 Cloud 서비스로 Bare Metal Cloud 또는 Bare Metal Server 라는 개념이 존재



※ 테넌트: 8번 테넌트(tenant) 장표 참조

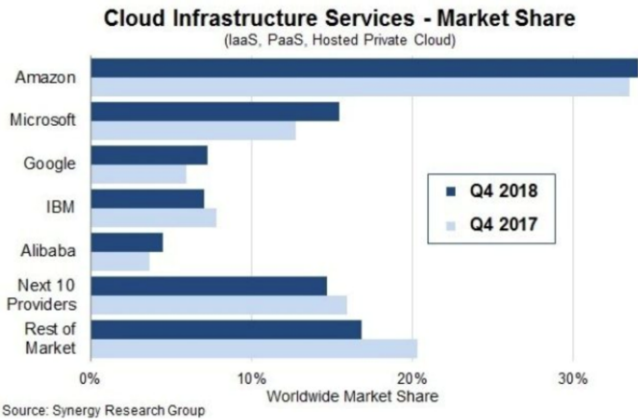
4

CSP,
CSC

- CSP(Cloud Service Provider) 정의: Cloud 서비스의 사용이 가능하도록 환경을 제공하고 사용량만큼 과금함
(과금 모델: 컴퓨팅 파워 사용량 단위, 데이터 인/아웃 사용량 단위, 가용 서비스 로드 단위 등 다양함)
- CSC(Cloud Service Customer) 정의: CSP가 제공한 Cloud 서비스를 이용. User의 의미로 CSU를 쓰기도 함

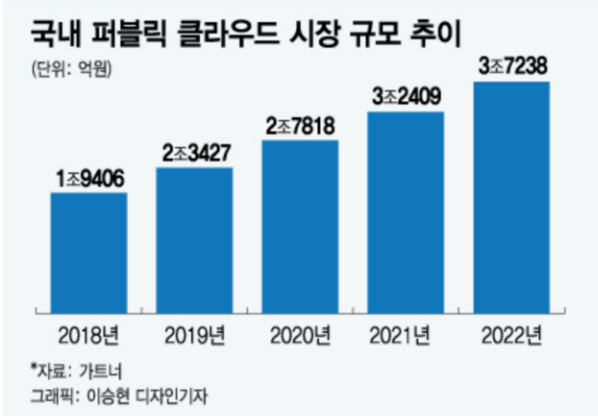
CSP 시장 점유율 ('2018 4분기)

- Cloud 시장에서 가장 높은 점유율을 가진 CSP는 Amazon에서 제공하는 AWS(약 35%)로 나타남
- 2위는 Microsoft에서 제공하는 Azure(약 15%)로 AWS와 Azure의 점유율이 50%를 차지하는 것으로 나타남



국내 퍼블릭 Cloud 시장 규모

- 올해 2조 3428억원으로 작년 대비 20% 이상 규모가 확대됨
- 국내 공공과 금융 Cloud 시장 규모도 확대 예상
 - 공공기관의 민간 Cloud 이용 가이드라인 -> 대국민 서비스
 - 전자금융감독 규정 개정안 시행(19년) -> 민감정보



※ 출처: <https://news.mt.co.kr/mtview.php?no=2019041009175256517>

MSP,
ISV

- ❑ **MSP(Managed Service Provider) 정의:** CSP에 소속되어 여러 기업들에게 네트워크, 애플리케이션, 시스템 등의 Cloud 구축·운영 서비스를 제공함. (실질적인 과금체계를 설계하고 운영함. 메가존, 베스핀글로벌 코리아 등)
- ❑ **ISV(Independent Software Vendor) 정의:** 컴퓨터 하드웨어나 플랫폼에서 실행 가능한 소프트웨어를 제작/판매

Cloud MSP의 필요 배경

CSP와 MSP, ISV

Cloud
환경의
다양화

- **Cloud 환경의 다양화**
 - 데이터 보안, 가용성, 비용 증가, 법적 요건 등의 이유로 프라이빗 Cloud 환경을 구축 필요성 증가
 - 요건에 맞게 설계한 하이브리드/멀티 Cloud 환경 증가

관리
복잡성

- **인프라 환경의 다양화**
 - 온프레미스, 프라이빗, 퍼블릭 Cloud 환경까지 인프라 환경이 다양해지면서 관리 복잡성 증가
 - Cloud 환경 구축 및 운영에 대한 전반적인 관리 필요성 증가

비용
절감

- **Cloud 환경 운영 비용 절감**
 - 사내 관리 인력 최소화
 - 인프라 관리 외 고객 서비스를 위한 작업 효율 증가

- **CSP에서 기술적 파트너십을 통해 자사의 플랫폼을 이용하는 부가 서비스 기업(MSP, ISV)들을 적극 모집 및 지원함**
- **CSP는 플랫폼 시장 점유율을 증가시키기 위해 다양한 native 솔루션 및 신기술을 개발, 홍보함**
- **MSP, ISV사는 해당 Cloud 플랫폼 이용자들을 대상으로 자사 제공 서비스를 홍보하고, 이익 증대를 도모하여, CSP와 서로 공존하는 관계를 형성함**



온라인하기고객지원한국어내 계정AWS 계정 생성

제품솔루션요금설명서학습하기파트너 네트워크AWS Marketplace더 알아보기Q

APN 파트너 유형

예시

APN 컨설팅 파트너

APN 컨설팅 파트너는 고객이 AWS에서 워크로드 및 애플리케이션을 디자인, 설계, 구축, 마이그레이션 및 관리하는 데 도움을 주는 전문 서비스 회사입니다. 컨설팅 파트너에는 SI(Systems Integrator), 전략적 컨설팅 업체, 대행사, MSP(Managed Service Provider), VAR(Value-Added Reseller)이 포함됩니다.

APN 컨설팅 파트너가 되는 방법에 대해 자세히 알아보십시오.

APN 기술 파트너

APN 기술 파트너는 AWS 플랫폼에서 호스팅하거나 AWS 플랫폼과 통합할 수 있는 소프트웨어 솔루션을 제공합니다. APN 기술 파트너에는 ISV(Independent Software Vendor), SaaS, PaaS, 개발자 도구, 관리 및 보안 공급업체가 포함됩니다.

APN 기술 파트너가 되는 방법에 대해 자세히 알아보십시오.

Cloud Native

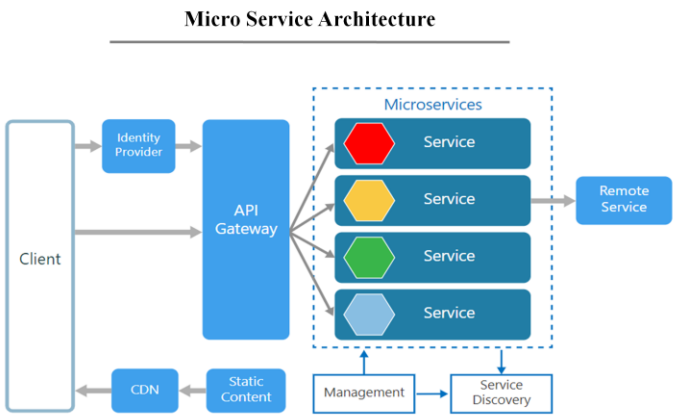
- 정의: Cloud 컴퓨팅 제공 모델의 이점을 활용하는 애플리케이션 구축 및 실행 접근 방법
(e.g. AWS native solution/service의 의미는 AWS에서 자체 제공하는 solution/service 프로그램을 의미함)
- 일반적으로 오픈 소스 코드를 사용, 컨테이너(Container)의 마이크로 서비스(Micro-services)로 구축하여 Cloud 컴퓨팅의 유연성과 확장성을 최대한 활용 가능함

Cloud Native 특징	Cloud Native 참조 아키텍처
• Container 단위로 관리 - 구동되는 애플리케이션은 독립적인 배포단위/자원관리 단위임 - Cloud Native application은 코드/컴포넌트 재사용 가능 • 동적 관리 - 중앙관리 방식의 스케줄과 관리로 서버 자원의 효율성을 높이고 자원관리와 운영 비용 절감 • Micro-services 중심 관리 - 의존관계가 명확하고 결합이 느슨한 모듈은 애플리케이션 전반에 민첩성과 관리 용이성 제공	Application Definition / Development Orchestration & Management Runtime Provisioning Infrastructure(Bare Metal/Cloud) • Container Native 환경 구축 - Meta data, 설정, 도구, 컨테이너 이미지 관리 도구 등 관리 • 동적관리를 위한 핵심 엔진 - 컨테이너 오케스트레이션 (Kubernetes, SWARM 등) 도구를 활용한 컨테이너 배포, 로깅, 모니터링 등 • 컨테이너 생성, 실행, 관리 - OCI(Open Container Initiative)¹⁾를 준수한 이미지 구동 보장 - 시스템 요구에 따라 적절한 런타임 선택적 사용 • 가상화(Private/Public) 인프라 - 가상화 플랫폼이 제공하는 API를 이용하여 클러스터 구성을 자동화함 - 구성의 오류와 작업을 효율적으로 가능함 • Cloud 네이티브 자원 인프라 - Bare Metal(舊 의미): 물리적인 인프라. 내부 또는 데이터 센터에 구성 - 가상화 인프라(Private Cloud) - 퍼블릭 Cloud(Public Cloud)

1) OCI (Open Container Initiative) : 컨테이너 기술에 관련된 공통적이고 최소화된 공개표준과 명세를 개발/확산하는 역할 수행을 위한 오픈 소스 단체

Micro-service Architecture

- 정의: 단일 어플리케이션을 상호 연결된 서비스 단위로 분할하여 서비스 간에 서로 통신하는 형태의 아키텍처
- 전통적인 어플리케이션 구축 모델인 모놀리식(monolithic) 아키텍처와 상반되는 개념
- Cloud Native에서 어플리케이션 개발 시 사용되는 핵심 기술 (APP의 Cloud 이전 시에도 적용하는 기술)



Micro Service 특징

- 각 서비스는 작고, 독립적이고, 느슨하게 결합되어 있음
- 작은 개발팀도 관리할 수 있는 개별 코드베이스로 개발함
- 서비스를 독립적으로 배포하여 어플리케이션 업데이트가 용이함
- 일부 결함이 발생해도 전체 어플리케이션 작동이 중단되지 않음

기타 구성 요소

Management	• 서비스 배치 • 실패 식별 • 서비스 부하 조정
Service Discovery	• 서비스 목록 관리 • 서비스 노드의 유지 관리 • 서비스 조회 및 엔드포인트 탐색
API Gateway	• 클라이언트의 진입점 • 서비스의 응답 집계 및 반환 • 인증, 로깅, SSL 종료, 부하분산 등 수행

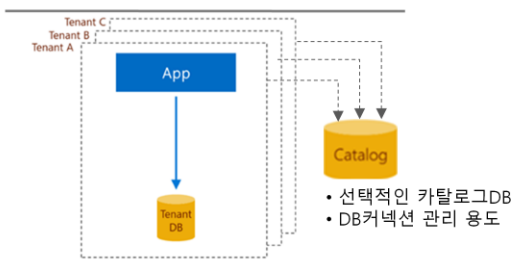
Tenant
(테넌트)

- 정의: SW 인스턴스에 대해, 특정 접근 권한을 공유하는 사용자 or 사용자 그룹

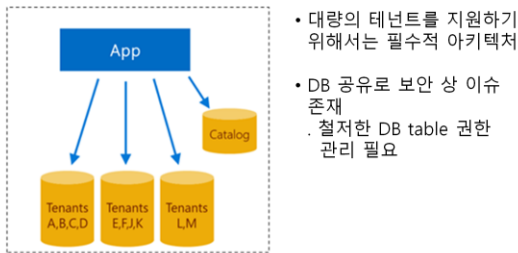
□ Cloud를 이용하는 다중 사용자가 환경 설정 등을 달리하여 APP을 공유하는 multi-tenancy SW Architecture의 이용 주체. Cloud는 서버 상 하나의 SW 인스턴스로 여러 개의 tenant에 서비스하는 개념임

□ tenant는 임대료를 대가로 APP 구성 요소에 대한 액세스 권한을 받고, 데이터를 저장하는 주체임

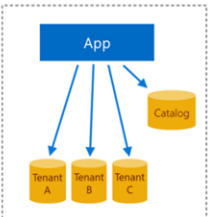
Case1) 독립APP – 단일 테넌트



Case3) 다중 테넌트 APP – 테넌트 공유 DB



Case2) 다중 테넌트 APP – 테넌트별 DB



※ 테넌트 비교

속성	독립 실행형 앱 (Case1)	테넌트별 데이터베이스 (Case2)	분할된 다중 테넌트 (Case3)
확장	중간 1-100s	매우 높음 1-100,000s	Unlimited 1-1,000,000s
테넌트 격리	매우 높음	높음	낮음. 단일 테넌트의 경우는 예외입니다(MT db에만 단독 으로 존재하는 경우).
테넌트 별 데이 터베이 스 비용	높음. 최대 크기에 맞게 조정됩니다.	낮음. 풀이 사용됩니다.	가장 낮음. MT DB의 작은 테 넌트용입니다.
성능 모 니터링 및 관리	테넌트별로 구분되는 경우만 해당	집계 + 테넌트별	집계. 단일 항목에 대해서만 테넌트별로 구분됩니다.
개발 복 잡성	낮음	낮음	중간. 분할로 야기됩니다.
운영 복 잡성	높음-낮음 개별적으로 는 단순하고 전체적으 로는 복잡합니다.	낮음-중간 권한으로 전체적 인 복잡성을 해결합니다.	높음-낮음 개별 테넌트 관리 가 복잡합니다.

※ 출처: <https://docs.microsoft.com/ko-kr/azure/sql-database/saas-tenancy-app-design-patterns>

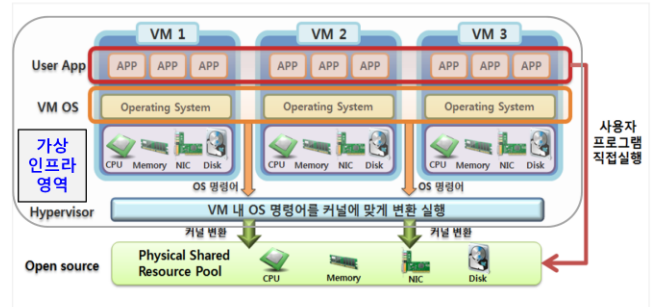
Virtual Infra-structure
(가상 인프라)

- 정의: 가상 환경을 제공하기 위해 필요한 하이퍼바이저, 가상머신, 가상자원 인터페이스 등으로 구성된 인프라
- 가상머신(VM: Virtual Machine)은 가상화 기술에 의해 논리적 시스템 자원(CPU, 메모리, 디스크, 네트워크 등)을 이용, 독립적으로 OS 운영환경이 실행되는 시스템
- 가상머신 OS, 사용자 App, 가상자원이 가상머신(VM)을 구성하며, 하이퍼바이저를 통해 하드웨어와 통신함

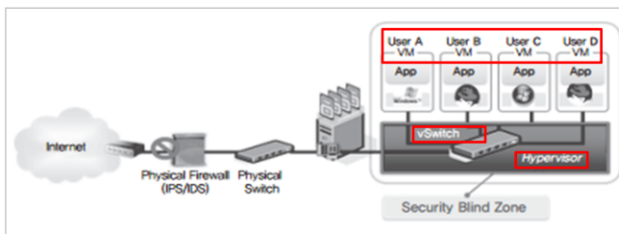
가상 인프라 개념



가상 인프라 세부 구조



가상 인프라 특징



■ Hypervisor

- 가상화 시스템 구조의 핵심
- 가상머신과 하드웨어 사이 위치
- 가상머신의 생성, 관리
- 자원 할당, 스케줄링

■ VM(Virtual Machine)

- 컴퓨터를 실행하는 OS
- 프로그램 에뮬레이터

■ vSwitch(Virtual Switch)

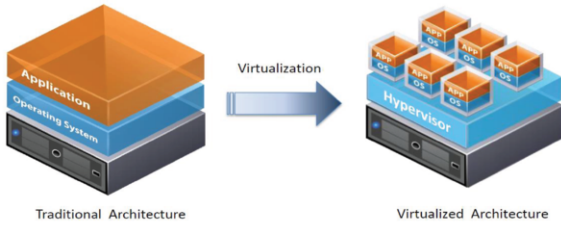
- 물리적 스위치와 연결되어 시스템 내-외부 스위칭

Hypervisor

- 정의: 가상 머신과 하드웨어 사이에 위치하여 다수의 가상 머신들 각각의 운영체제가 구동될 수 있도록 논리적으로 독립된 환경을 제공하는 프로그램. VMM(Virtual Machine Manager/Monitor)이라고 하기도 함
- Bare-metal : H/W 상에서 바로 동작하는 형태로 고성능의 가상화를 제공(Xen, VMWare ESXi, MS Hyper-V 등)
- Hosted Style : 개인컴퓨터의 운영체제 상에 응용프로그램의 형태로 인스톨되어 가상머신을 구동할 수 있는 환경 (VMWare Workstation, VMWare Fusion, Parallels Desktop, Oracle VirtualBox 등)

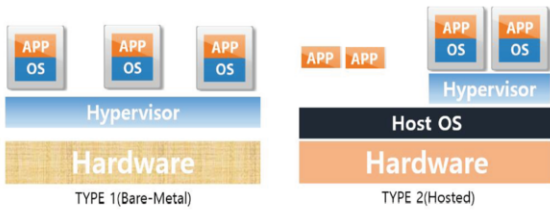
가상화 아키텍처 개념

[가상화 아키텍처]



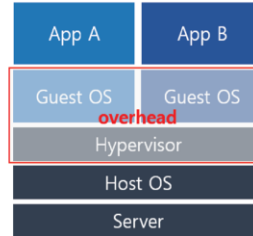
Hypervisor 2-style

[하이퍼바이저 타입]



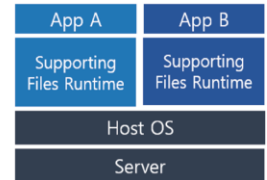
VM과 Containers 비교

VM



[Hypervisor]

Containers



[Docker]

- VM : 단일 시스템에서 여러 운영체제가 동시에 실행
- Container : 서로 다른 어플리케이션 별로 공간을 격리

※ Docker의 특징

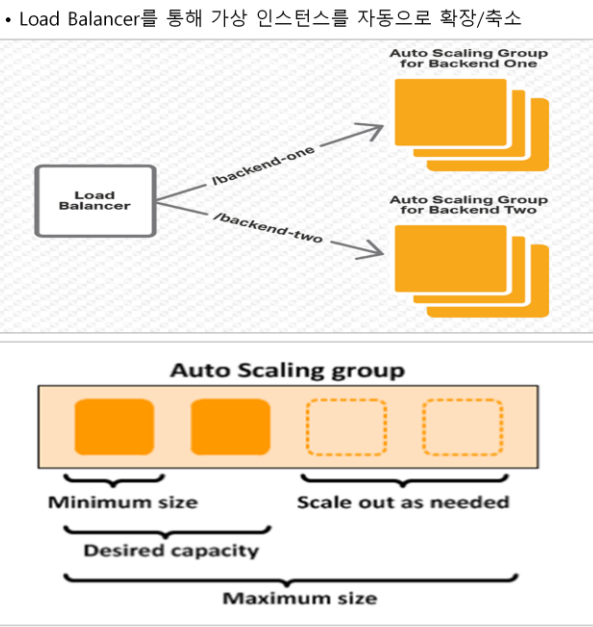
- 이미지와 컨테이너로 구성
- Image : 서비스 운영에 필요한 프로그램, 라이브러리 포함
- Container : 이미지를 실행한 상태
- Hypervisor와 Guest OS를 설치해야 하는 가상환경보다 간편

* KVM이 bare-Metal type인지는 추후 판정

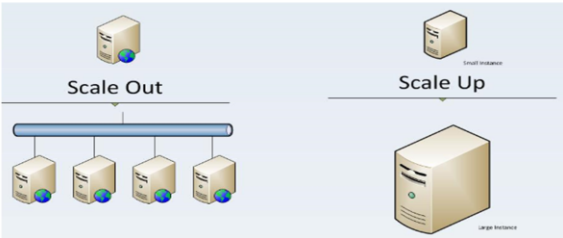
Auto Scaling

- 정의: Cloud 컴퓨팅 서비스에서 부하량과 사용량에 맞게 탄력적으로 컴퓨팅 자원을 늘리거나 줄이는 기능
- Scale Up : CPU 변경 또는 RAM의 추가 등 하드웨어 성능을 높여서 처리 능력 향상
- Scale Out : 하나의 장비에서 처리하던 일을 여러 장비에 나눠서 처리할 수 있도록 설계 변경하여 처리 능력 향상

Auto Scaling 구조



Scale Out VS. Scale Up



※ 가상화 환경에서는 Scale Out 사용

구분	Scale Out	Scale Up
확장성	수평확장, 지속적 확장 가능	수직확장, 성능 확장에 한계
장애 영향도	여러대의 서버에 분산되어 처리되어 영향도 적음	한대의 서버에 부하가 집중되어 장애시 영향도 큼
주요기능	Query-off Loading, Queue, In Memory Cache, Sharding,	고성능 CPU, Memory 확장 SSD
주요용도	분산처리시스템/Global 웹 어플리케이션	고성능 Legacy 어플리케이션
장점	비용저렴, 점진적 증가 가능	구축 및 관리 용이
단점	설계/구축/관리 비용 증가	단계적 증가가 어려움

Cloud 서비스
배포 모델

□ 모델 유형

- Cloud 서비스 배포 모델 유형은 Cloud 배포 장소와 서비스를 제공/사용 하는 주체에 따라서 **Public Cloud / Private Cloud / Hybrid Cloud / Multi Cloud(멀티 Cloud)**로 구분됨
※ DaaS, CaaS 등 서비스의 한 종류와는 구분하여 사용해야 함

구분	Public Cloud	Private Cloud	Hybrid Cloud	Multi Cloud
특징	- 다수의 대중을 위하여 인터넷 기반으로 운용 - Cloud 인프라가 이용자 에겐 추상화되어 단순하게 보임(e.g. S3) 책임 공유 모델	- 대외에 오픈되지 않고 내부망 위주로 운용 SDDC (SDC/SDN/SDS/Restful API 등) ※ HCI (Hyper-Converged Infrastructure) ※ OpenStack ⇒ VMWare - 보안 및 신뢰성 제고 PaaS 구축 및 운영 - 네트워크 대역폭, 인터넷 접속 제약 없음 - 서비스 수준 합의/관리 (SLA : Service Level Agreement / Mgmt.) 인프라 관리 편리성	- Public + Private Cloud - Public + On-prem.을 결합하여 운용 ※ On-prem. 상면(IDC등) + CSP's Cloud SW 구축 ⇒ AWS OutPost, Az Stack, NCP Neurocloud, NHN Private Deck 등 - Public Cloud와 Private Cloud의 이점 활용 (AWS - VMware 연계) - 높은 수준의 유연성 - 다양한 배포 옵션 제공 - 탈출 계획 이행 유리	- 멀티 Public CSP로 구성하여 운용 - On-prem. 연계성이 hybrid 대비 상대적으로 다소 약한 니앙스 포함 - 고 가용성 - 높은 수준의 유연성 - 업체 종속성 탈피 - 비용 효율성 - 탈출 계획 이행 유리
장점	- 규모의 경제를 통한 비용 관리 (전략적 결정) - 이용한 만큼 지불 서비스 구현 속도 향상 높은 수준의 탄력성 손쉽게 이용 가능한 고 품질 응용 프로그램	- 규모의 경제를 통한 비용 관리 (전략적 결정) - 이용한 만큼 지불 서비스 구현 속도 향상 높은 수준의 탄력성 손쉽게 이용 가능한 고 품질 응용 프로그램	- Public Cloud와 Private Cloud의 이점 활용 (AWS - VMware 연계) - 높은 수준의 유연성 - 다양한 배포 옵션 제공 - 탈출 계획 이행 유리	- 멀티 Public CSP로 구성하여 운용 - On-prem. 연계성이 hybrid 대비 상대적으로 다소 약한 니앙스 포함 - 고 가용성 - 높은 수준의 유연성 - 업체 종속성 탈피 - 비용 효율성 - 탈출 계획 이행 유리
단점	- Cloud 서비스 제공자에 대한 통제 권한 부족 - Private Cloud에 비해 상대적으로 취약한 보안 (인터넷, 하드웨어 공유 문제 등)	# Public Cloud 대비 - 장비, 하드웨어 비용 ↑ - 가상화 기술 비용 ↑ - 상대적 낮은 탄력성 - 운용이 복잡함 ↑	- Public Cloud와 Private Cloud간 연계 시 보안 통제의 어려움 - 높은 구축 비용 및 운용의 어려움 - 네트워크 구성이 복잡함	2개 이상의 Cloud 제품을 사용하므로 운용이 복잡하고 어려움 (기술, 인력 이슈)

Checkpoint

- Cloud에서의 비용 유형

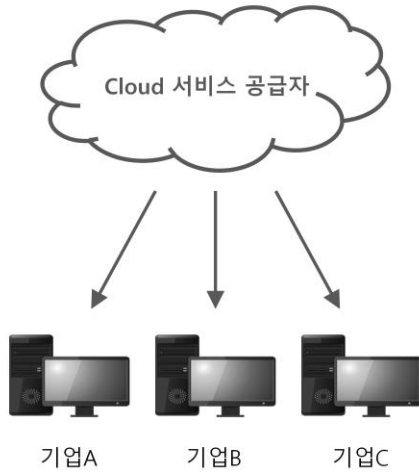
. **CapEx(자본 비용)** : 온프레미스 환경에서 사장님께 결제받아야 하는 수천만원짜리 장비들이 전부 자본비용에 해당한다. 해당 장비들은 구매 후 자산으로 분류되며 시간에 따라 감가상각된다.

. **OpEx(운영 비용)** : 클라우드 서비스는 전부 OpEx에 해당한다. 초기 비용이 없으며 사용하는 만큼 지불하는 종량제 시스템이다.

Public Cloud
(공용 Cloud)

□ 정의

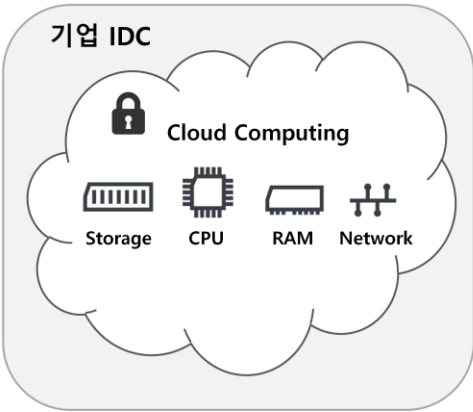
- Cloud 서비스 제공자(CSP)가 구축한 서버, 스토리지 등의 IT 인프라를 기업들이 사용료를 내고 이용하는 Cloud 운용 방식



- 모든 하드웨어, 소프트웨어 및 기타 지원 인프라를 Cloud 서비스 제공자(CSP)가 소유하고 관리
- 다른 기업 또는 Cloud 이용자와 동일한 하드웨어, 저장소 및 네트워크 장치를 공유 (※ 데이터는 분리되어 있음)
- 저렴한 서비스 비용으로 인한 경제성, 시스템 유지보수나 장애 대응을 위한 효율성과 같은 Cloud 서비스의 장점에 최적화

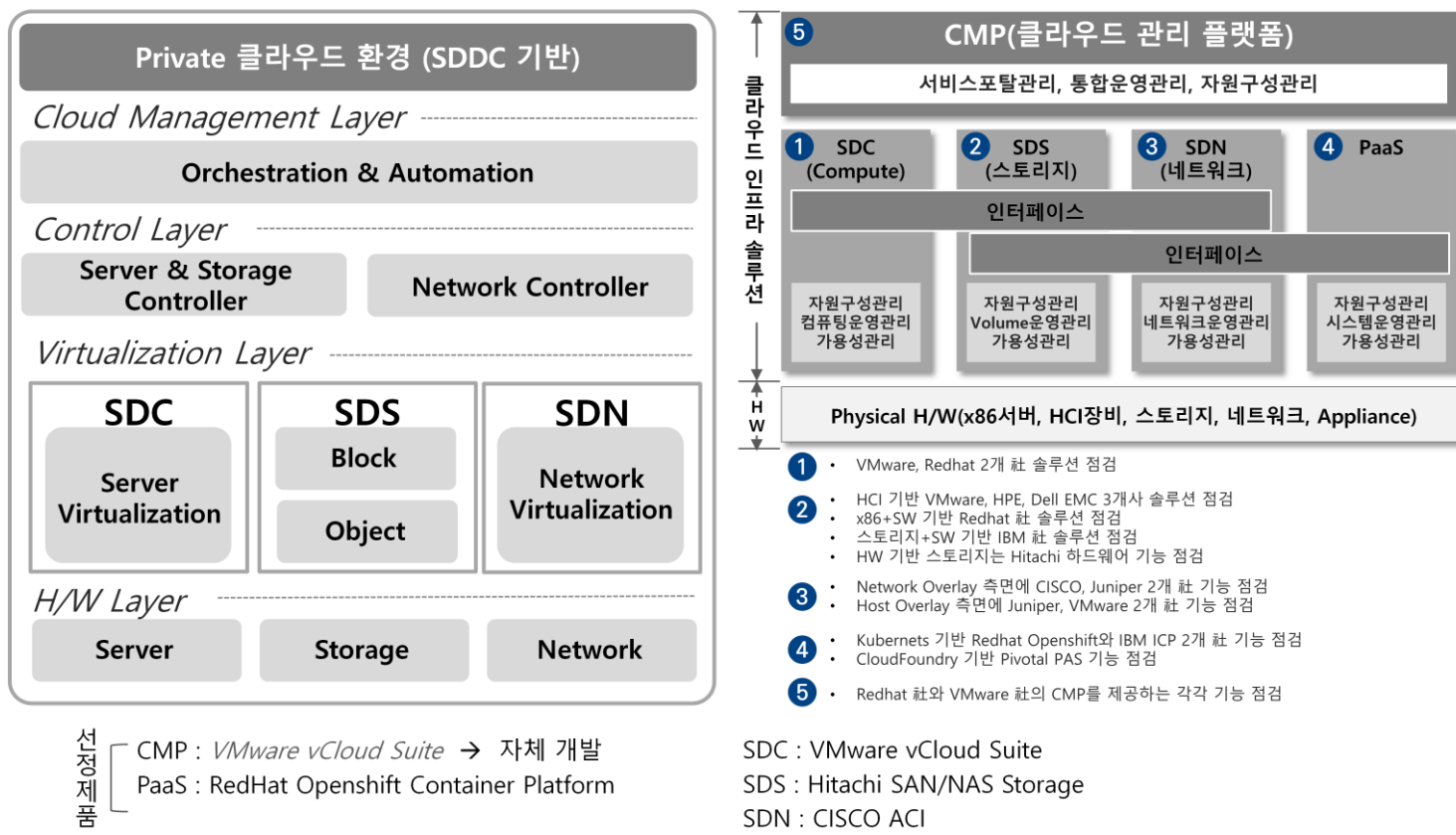
Private Cloud
(사설 Cloud)

- 정의
- 기업들이 자체적으로 데이터센터 안에 직접 Cloud 환경을 구축해 사용하는 Cloud 운용 방식



- Cloud 플랫폼이 기업 내부 혹은 3rd party 데이터 센터에 독립적으로 구축되므로, 기업이 인프라에 대한 직접적인 통제권한을 가짐
- 인프라 환경을 원하는 형태로 이용자의 요구사항에 맞게 직접 구성할 수 있음 (Compliance 직접 대응)
- 내부 통제가 가능하며, 기업의 정보 자산을 내부에 유지 보관하여 보안성을 자체 확보할 수 있음

Private Cloud는 가상화, 클라우드 솔루션을 직접 구축하여야 하며, 유관 서비스도 직접 개발해야 함
Private Cloud SDDC(Software Defined Data Center) HCI(Hyper Converged Infrastructure)



※ Checkpoint

- Private Cloud 이해

... [컴퓨터월드] 퍼블릭 클라우드 사용자가 가장 먼저 체감하는 장점은 컴퓨팅, 스토리지, 네트워크 자원을 모두 소프트웨어(SW) 기반으로 가상화해 필요한 자원을 클릭 몇 번으로 프로비저닝해 쓸 수 있다는 것이다. 이러한 '소프트웨어 정의 데이터센터(Software-Defined Data Center; SDDC)' 기술은 최근 프라이빗 클라우드 측면에서도 HW 인프라 기업들이 그 중요성을 강조하고 있다.

SDDC는 SW로 관리 및 통제되는 컴퓨팅(SDC, 서버 가상화), 소프트웨어 정의 네트워킹(SDN), 소프트웨어 정의 스토리지(SDS) 기술을 모두 활용하며, 통합적이고 자동화된 관리를 위한 레스트풀API(Restful API) 지원도 'SW 정의'의 한 요소라고 할 수 있다.

그렇다면 SDDC 기반의 프라이빗 클라우드 구축에 가장 적합한 HW는 무엇일까. 물론 기존 서버, 스토리지, 네트워크를 갖고도 SDDC를 구축할 수는 있다. 그러나 각각의 인프라가 분리돼있는데다 이를 구매해 테스트하기까지 소요되는 시간이 수십 일에 달한다는 점에서, 퍼블릭 클라우드의 최대 장점인 데이터센터 구축 및 관리의 복잡성 제거 측면은 따라잡기가 쉽지 않다.

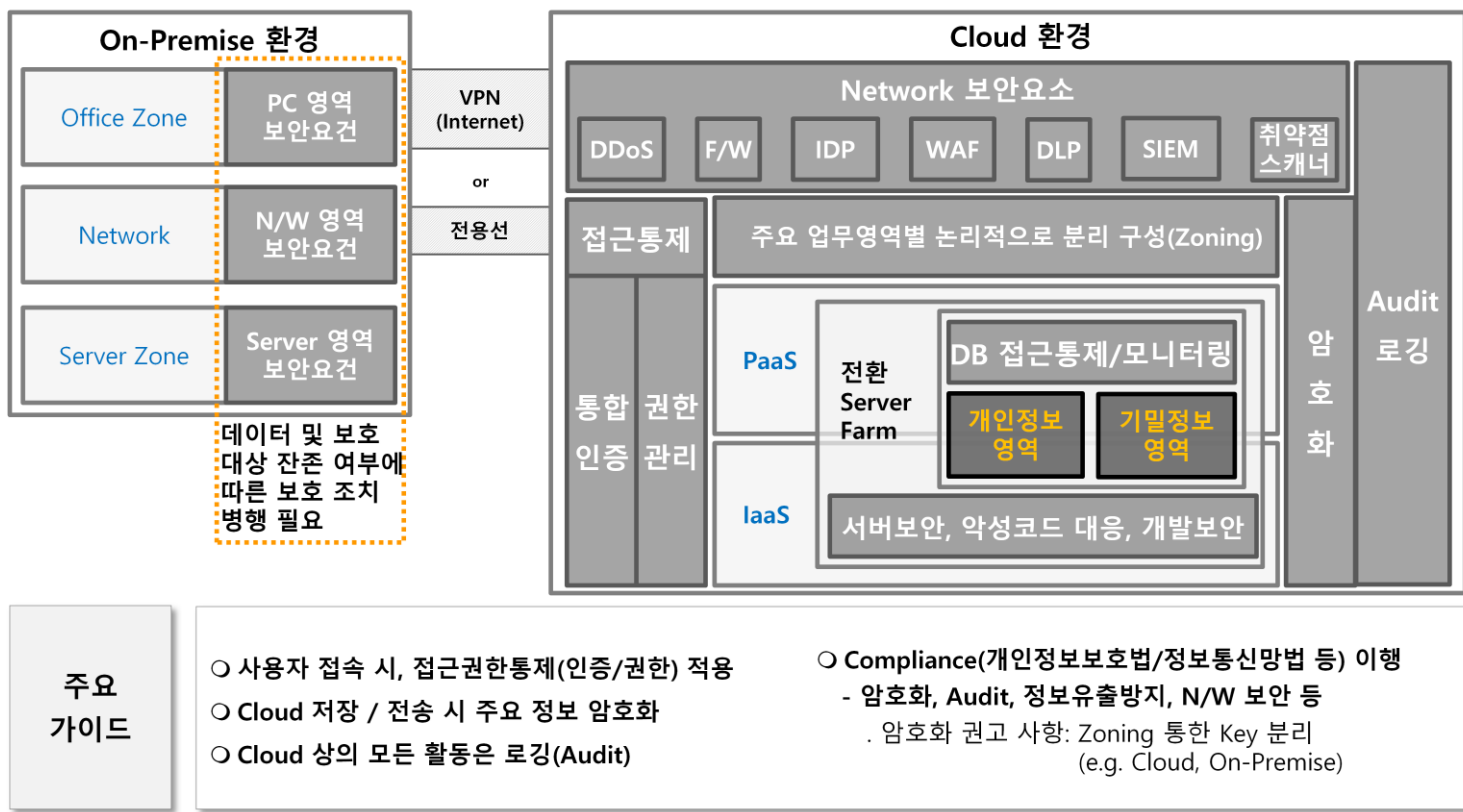
하이퍼컨버지드 인프라(Hyper-Converged Infrastructure, HCI)는 이를 해결해준다. HCI는 표준 x86 서버 및 스토리지, 네트워킹 스위치가 하나의 장비에 집적돼 있으며, 이를 SW 정의 기술을 바탕으로 통합 관리할 수 있다. 이에 상면 감소로 인한 총소유비용(TCO) 절감은 물론 모듈 방식을 기반으로 인프라의 손쉬운 스케일아웃이 가능해 도입과 테스트에 걸리는 시간이 불과 몇 시간에서 길어야 며칠에 불과하고, 퍼블릭 클라우드처럼 필요에 따라 작게 시작해 인프라를 확장해나갈 수 있다는 장점도 갖는다.

글로벌 시장에서 뉴타닉스가 HCI 개념을 처음 선보인 이래 델EMC, HPE, 시스코 등 쟁쟁한 인프라 HW 기업들이 속속 이 시장에 뛰어들고 있는 이유다. ...

* 출처: <http://www.itdaily.kr/news/articleView.html?idxno=91133>

[Backup] Private Cloud 아키텍처 #2

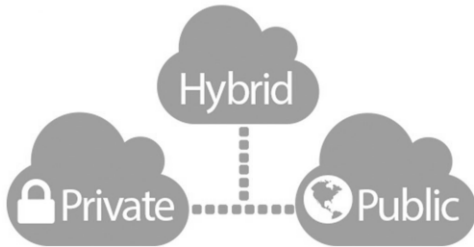
클라우드 서비스 배포 모델에 따라 On-Premise 수준 이상의 보안환경을 구성하되, Cloud 환경에 맞게 컴플라이언스 보안 요건, 보안강화 요건 등의 적절한 적용 수준 적용 필요



Hybrid Cloud
(하이브리드
Cloud)

□ 정의

- On-Premise 인프라 또는 Private Cloud를 Public Cloud와 결합하여, 두 분야의 이점을 얻을 수 있도록 구성된 Cloud 운용 방식

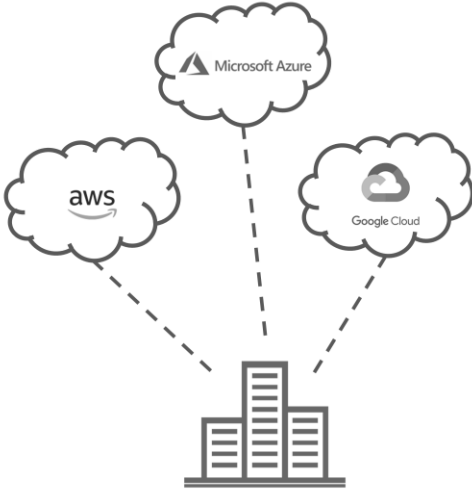


- Private Cloud와 Public Cloud간 데이터와 응용 프로그램이 이동할 수 있으므로 유연한 서비스 운용이 가능함
- 예를 들어 공용 서비스를 위한 WEB/WAS 등의 서버는 Public Cloud를 이용하여 Cloud의 장점(비용 및 확장성)에 최적화시키고, 보안이 중요한 서버(DB 등)들은 Private Cloud로 구성하여 외부의 직접적인 접속을 차단

**Multi Cloud
(멀티 Cloud)**

□ 정의

- 2곳 이상의 Cloud 서비스 제공자(CSP)가 제공하는 2개 이상의 Public Cloud 또는 2개 이상의 Private Cloud로 구성하여 하나의 서비스를 운영하는 방식



- 기능, 가격, Compliance 등 특정 Cloud 업체에 얽매이지 않고 장기적인 로드맵 구성 가능
- 컴퓨팅 자원을 분산하고, 다운타임과 데이터 손실 위험을 낮출 수 있음
- 각 Cloud 플랫폼의 장점만을 결합하여 운용할 수 있음
- 한 업체의 데이터센터 대규모 장애를 대비하여 이중화 구성 가능

Cloud Computing

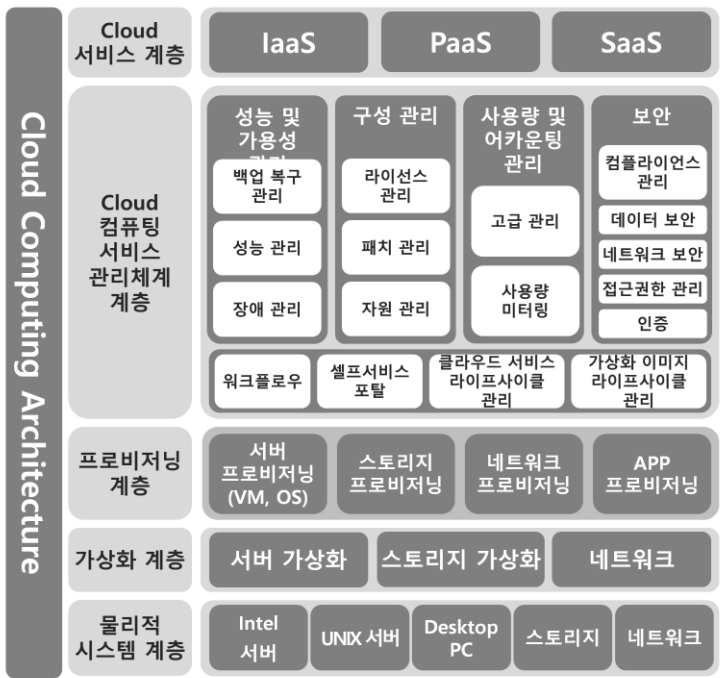
- 정의: 서버, 스토리지, 데이터베이스, 네트워크, 소프트웨어 등의 컴퓨팅 서비스를 공유 네트워크("Cloud")를 통해 제공하는 것
- 목적: 비즈니스 규모에 맞춰 합리적인 비용으로 비즈니스 서비스를 효율적으로 운영할 수 있도록 환경 제공

Cloud Computing 주요 특성

구분	설명
On-demand self-service	- 고객은 제공사의 도움 없이 필요에 따라 자동적으로 서버 시간과 네트워크 저장소와 같은 컴퓨팅 기능을 단독으로 준비할 수 있음
Broad network access	- 모든 기능들은 네트워크를 통해 이용 가능함 - 기기종 클라이언트 플랫폼(예: 모바일폰, 태블릿, 노트북, 워크스테이션)에서 접근 가능함
Resource pooling	- 고객 요구사항에 따라 물리/가상화된 자원들을 멀티 테넌트 모델을 사용하는 복수의 고객들에게 제공하기 위해 공유 관리함 - 고객이 제공받는 자원의 정확한 위치에 대한 정보 없이 장소 독립적으로 운용 가능함
Rapid elasticity	- 기능은 탄력적으로 준비되고 배포 가능함 - 자동으로 요구에 비례하여 빠르게 규모를 확장 또는 축소 가능함 (Auto Scaling)
Measured service	- 서비스 유형: 저장소, 프로세싱, 대역폭, 활성계정 등에 맞춰 측정 기능을 활용하여 리소스를 자동으로 제어하고 최적화함 - 자원 사용은 감시, 제어, 보고되어 서비스 이용자와 제공자 모두에게 투명성을 제공함

※ 출처: The NIST Definition of Cloud Computing(NIST, 2011)

Cloud Computing Architecture



※ Check Point

- Cloud : 빌려 쓸 수 있는(tenantable) SW 체계를 쉽게 접근 가능하도록 구현해 놓은 통합 인프라 플랫폼
- 빌려 쓰는 SW 개념의 연원
 - . On premise – ASP – SaaS – **Cloud**
(테넌트별 독립HW) (**가상화, Software defined tech.**)
- Cloud vs 가상화, 우리회사는 서버 가상화를 사용하고 있는데 굳이 Cloud를 써야 하나요?
- 가상화: 서버 / 스토리지 / 네트워크 / APP / Desktop(e.g. VMWare Horizon – Cloud PC)

On-Premise

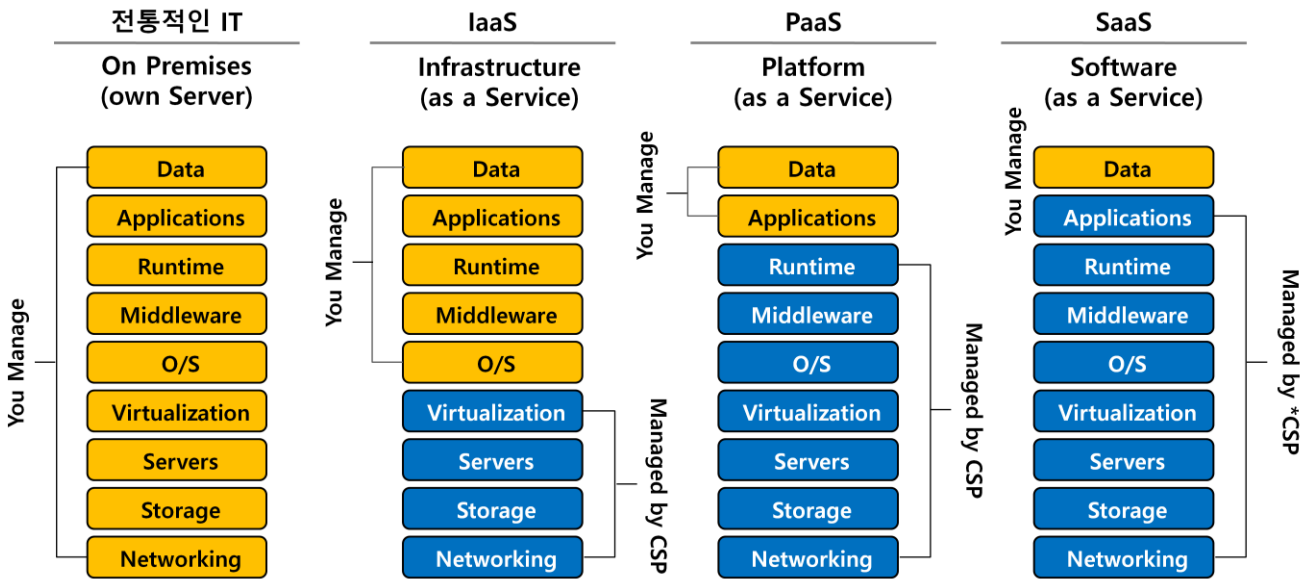
□ 정의: 소프트웨어 등을 (원격) 공유 환경이 아닌 자체적으로 보유한 HW 자산에 직접 설치해 운영하는 방식을 의미함

□ Cloud 컴퓨팅에 반대되는 개념으로 전통적인 기업 인프라 구축 방식을 의미함

구분	On-Premise	Cloud Platform(SaaS)
호스팅	• 기업 자체 서버를 통해 솔루션이 제공됨 • 사내 네트워크 등 접근에 대한 제한이 있음	• 서비스 공급 업체 서버를 통해 솔루션 제공됨 • 인터넷에 연결된 모든 장치에서 접근 가능함
비용	• 하드웨어, 솔루션 등 도입 비용이 많이 필요함	• 월간 또는 연간으로 솔루션 사용 비용을 지불함
구축 시간	• 인프라, 솔루션 설치 등 구축 시간이 오래 걸림	• 준비된 환경을 배포만하기 때문에 구축 시간이 짧음
사용자 정의	• 기업 환경에 최적화된 솔루션 사용이 용이함	• 사용자 요구에 맞는 맞춤형 솔루션 설치가 어려움
유지보수	• 버그, 최신패치 등 유지보수 비용이 발생함	• 버그, 최신패치 등을 공급자가 유지보수를 관리함
스케일	• 스케일 변경 시 많은 비용과 시간이 필요함	• 자원 사용량을 즉각적으로 변경하여 쉽게 이용함

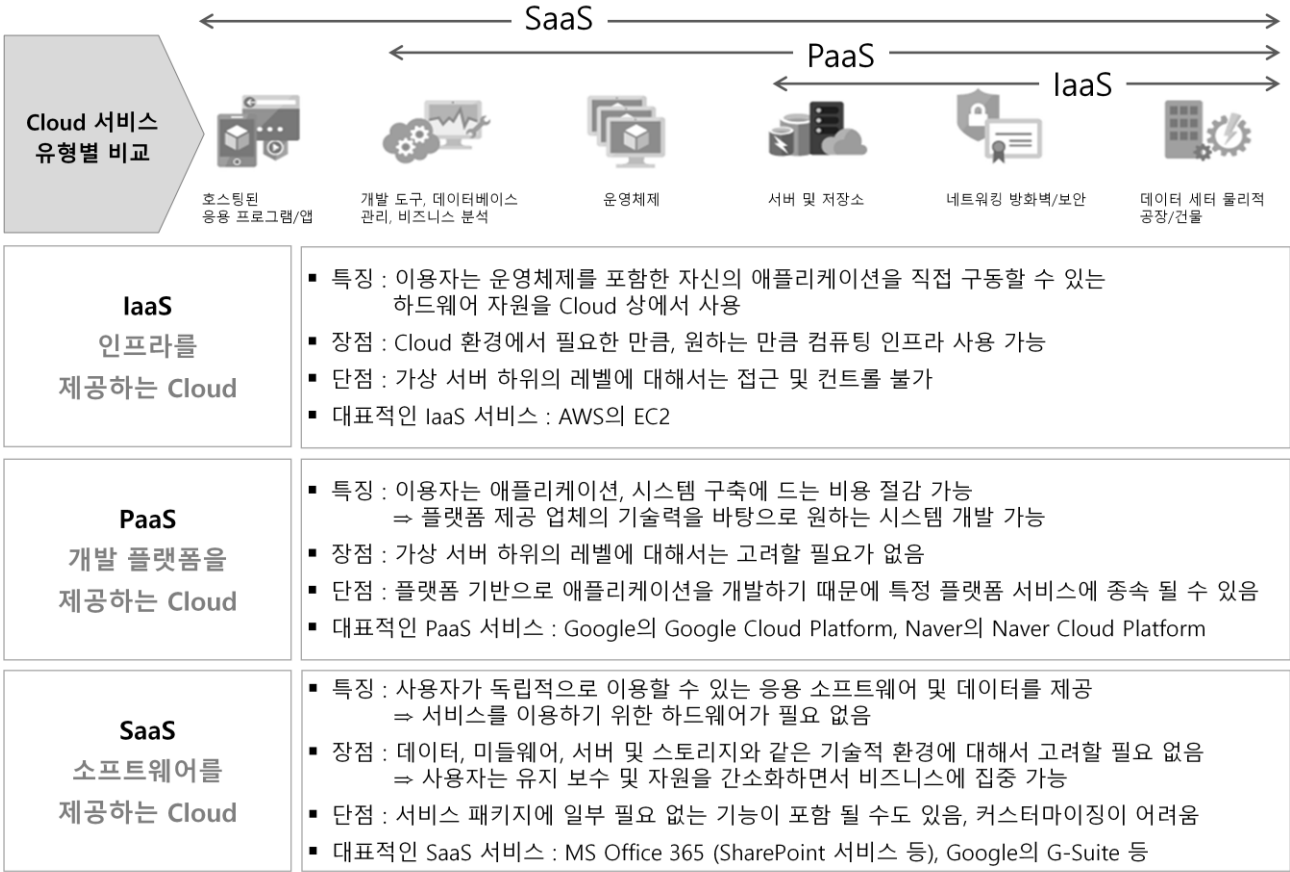
Cloud
서비스 모델

- 모델 유형
- Cloud 컴퓨팅 서비스 모델 유형은 Cloud에서 제공하는 서비스 범위에 따라 인프라 서비스(IaaS), 플랫폼 서비스(PaaS), 소프트웨어 서비스(SaaS)로 구분됨



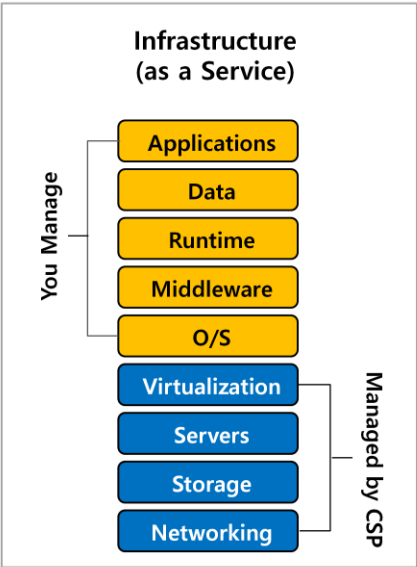
- 얼마만큼 사용자가 관리하고 얼마만큼 Cloud에서 제공받는가에 따라 구분 됨(책임공유모델)
- 노란색의 You Manage는 사용자 관리영역, 파란색의 Managed By CSP는 CSP 업체 관리영역

* CSP : Cloud Service Provider



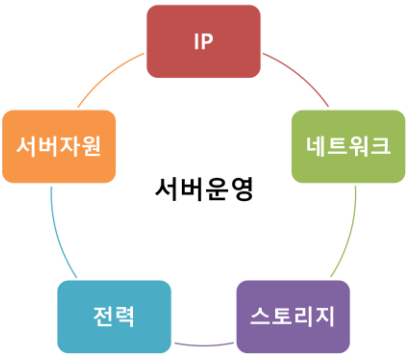
IaaS

- 정의
- 서버, 스토리지, 네트워크 같은 하드웨어 자원 혹은 하드웨어의 컴퓨팅 능력만을 필요에 따라 인프라 자원으로 사용할 수 있게 서비스를 임대 및 제공하는 형태



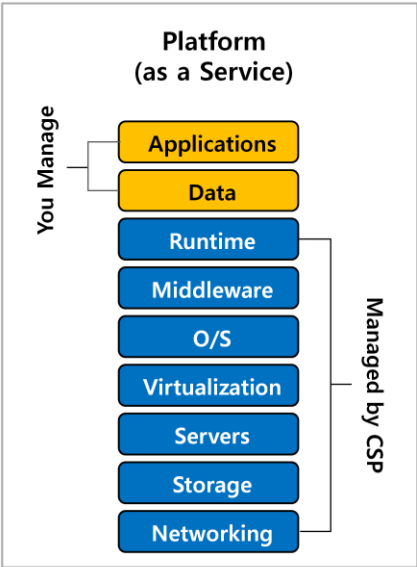
* CSP : Cloud Service Provider

- 인프라 구축을 가상의 환경에서 쉽고 편하게 이용할 수 있게 서비스 형태로 제공함



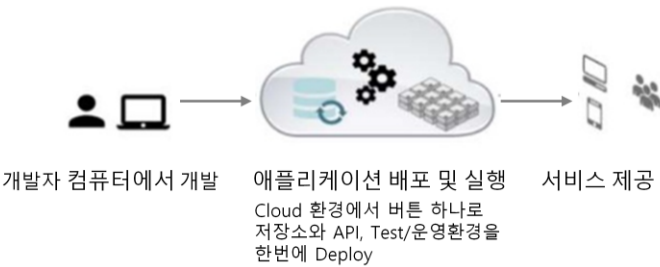
- 정의
- PaaS

 - 소프트웨어나 서비스를 개발하기 위해 필요한 **환경(Platform)**을 임대 및 제공하는 형태
 - 소스코드 배포/실행 환경을 별도 구성없이 Cloud 상에서 제공하는 서비스
 - 개발자 제공 플랫폼이기에 보안성심의(개발보안)과 연계되어 사업이 수행될 가능성이 높음



* CSP : Cloud Service Provider

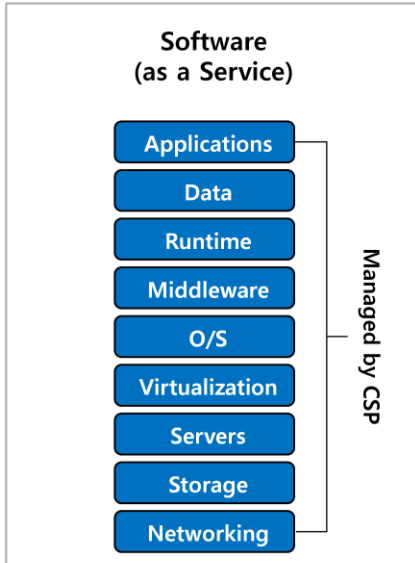
- 서비스를 개발할 수 있는 안정적인 환경(Platform) 제공
- 애플리케이션을 개발할 수 있는 API 제공
- 서비스를 구성하는 컴포넌트 및 호환성 제공 서비스 지원



SaaS

□ 정의

- Cloud 환경에서 동작하는 응용프로그램을 서비스 형태로 제공하는 것
- Cloud 컴퓨팅 최상위 계층에 해당하는 것으로 다양한 애플리케이션이나 소프트웨어를 필요할 때마다 온디맨드(On-Demand) 방식으로 이용할 수 있도록 하는 서비스



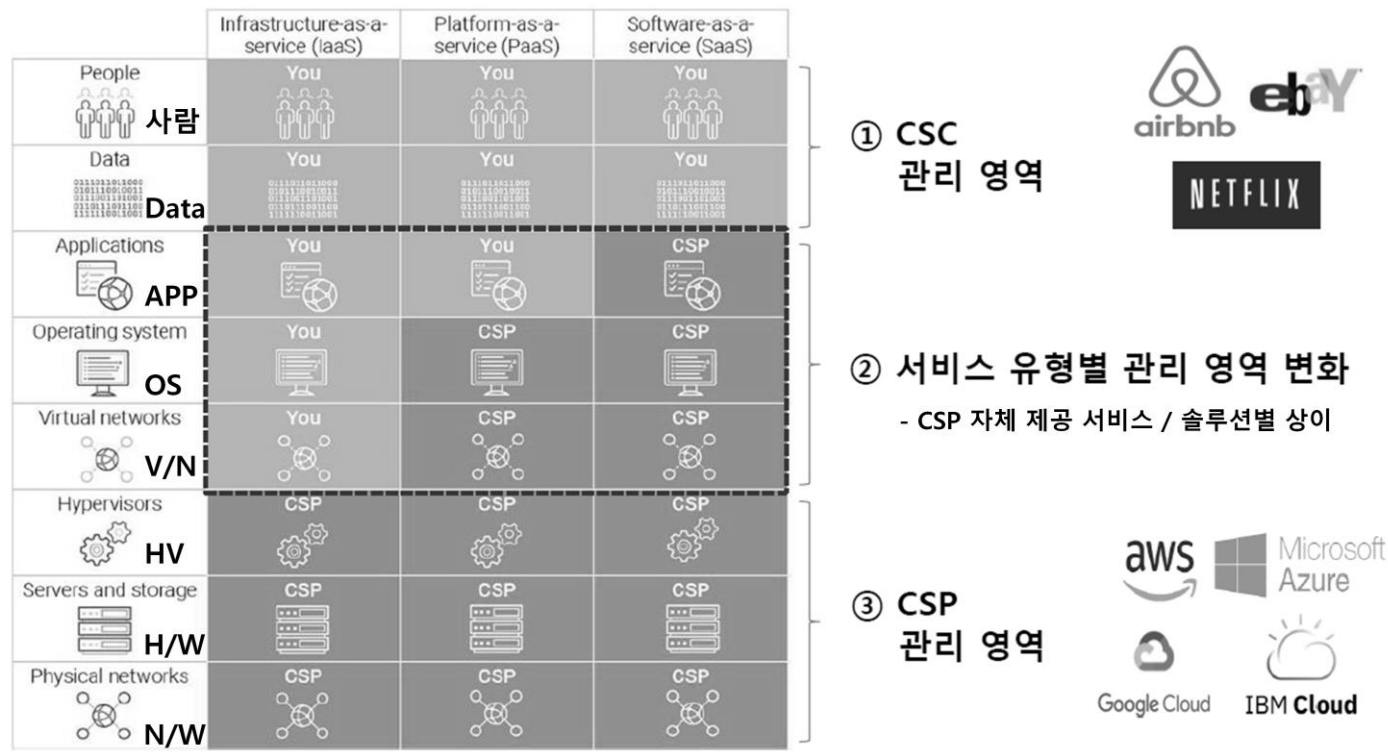
* CSP : Cloud Service Provider

- 모든 서비스를 Cloud에서 제공하므로
사용자는 별도의 설치 없이 소프트웨어를 사용할 수 있음
- 중앙에서 해당 소프트웨어를 관리하므로
사용자가 업그레이드나 패치 작업 수행 할 필요 없음



[Backup] Cloud 보안 책임공유모델(Shared Responsibility Model) (1/3)

Cloud 보안 위험을 대응하기 위해서는 고객, 서비스 제공자가 책임져야 할 각자의 영역을 이해해야 함

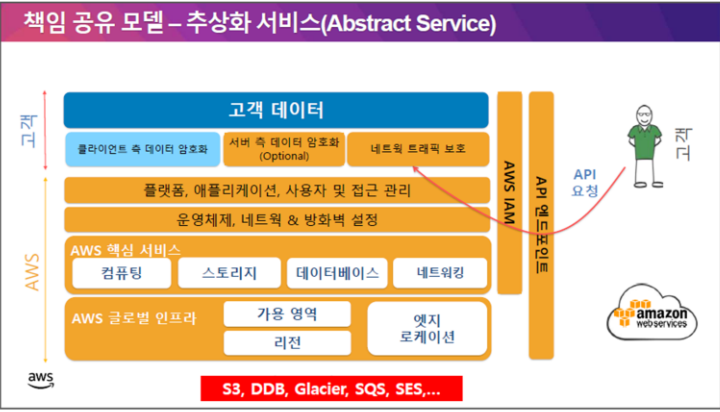
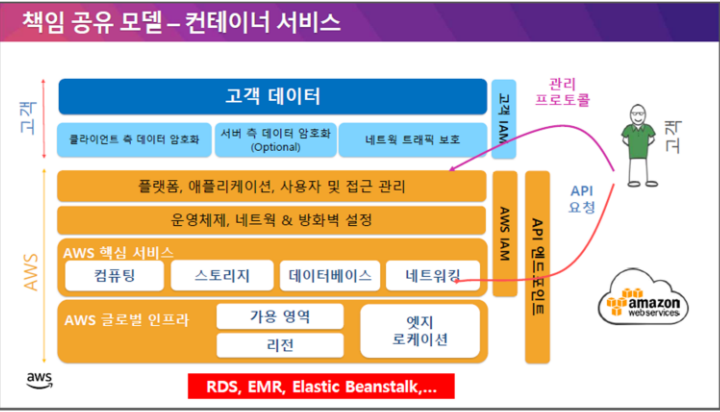
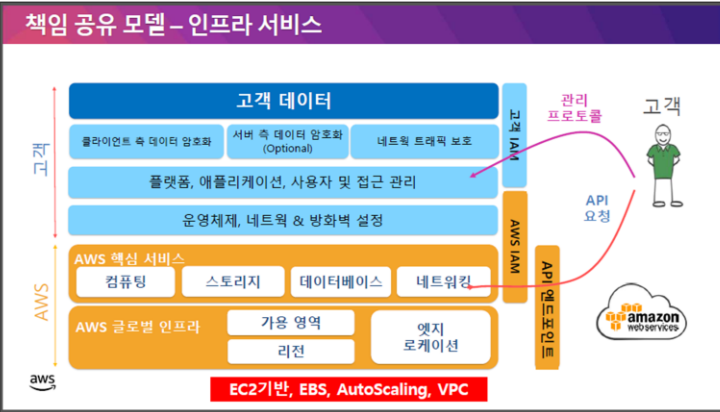


※ Check Point

- 책임공유모델의 출발: “접속 계정의 password 유노출을 CSP에서 책임질 수는 없다.”

[Backup] Cloud 보안 책임공유모델(Shared Responsibility Model) (2/3)

AWS의 경우, 인프라 / 컨테이너 / 추상화 서비스별 책임 영역이 상이함 (e.g. AWS EC2 서비스 - IaaS 유사)



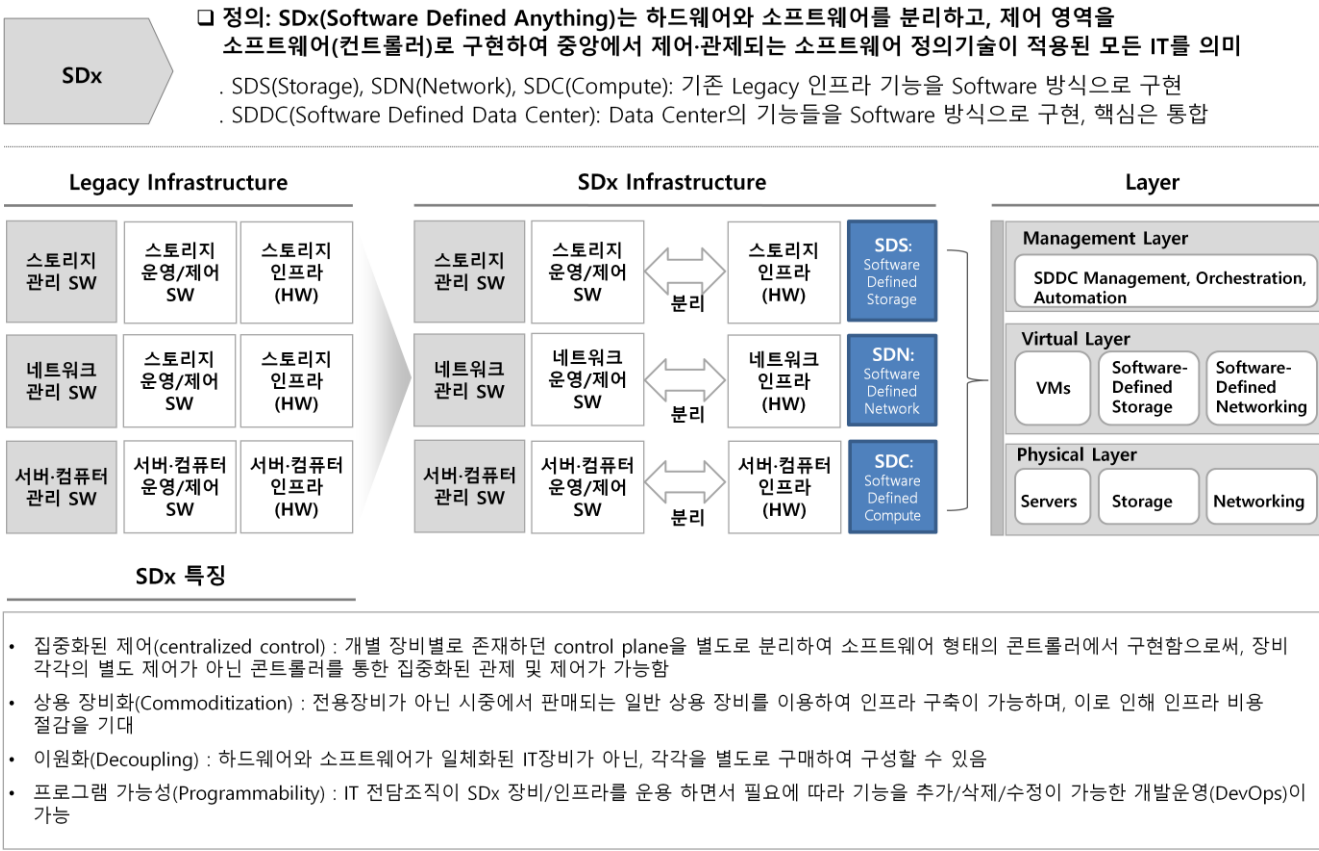
※ Check Point

- 추상화: 중요한 특징을 찾아낸 후 간단하게 표현하는 것
 - . 추상화는 필요한 부분, 중요한 부분을 통합하여 하나로 만드는 것
 - . 대표적인 예: 스마트폰 상의 아이콘
- : 추상화는 여러 가지 요소들을 하나로 통합하는 방향성을 가지고 있음.
- 한 가지 문제를 여러 가지로 쪼개서 나눠보는 '문제 분할'과 반대의 개념임.
- 추상화를 이용하여 핵심적인 것만을 남겨 표현하게 되면, 복잡한 내용도 한 눈에 알아볼 수 있어 이해하기 쉽다는 장점이 있음.

구분	AWS	이용자
① 규정 준수 측면	▪ 시설 ▪ 물리적 보안 ▪ 컴퓨트 인프라 ▪ 스토리지 인프라 ▪ 네트워크 인프라 ▪ 가상화 레이어 ▪ 정립된 서비스 엔드포인트 ▪ 다양한 IAM 기능	▪ 네트워크 환경설정 ▪ 시큐리티 그룹 ▪ OS 방화벽 ▪ 운영체제 ▪ 어플리케이션 ▪ 적절한 서비스 환경설정 ▪ 자격 인증 및 계정 관리
② PaaS 측면	▪ 근원 서비스 - 네트워킹, 컴퓨트, 스토리지 ▪ AWS 글로벌 인프라 ▪ AWS API 엔드포인트 ▪ 운영체제 ▪ 플랫폼/어플리케이션	▪ 고객 데이터 ▪ 방화벽 / VPC ▪ 고객 IAM(유저, 그룹, 룰, 정책) ▪ 고 가용성 ▪ 데이터 보호(전송, 대기, 백업) ▪ 확장성
③ SaaS 측면	▪ Foundational Services 근원 서비스 ▪ AWS 글로벌 인프라 ▪ AWS API 엔드포인트 ▪ 운영체제 ▪ 플랫폼/어플리케이션 ▪ 데이터 보호(대기 시 서버 측 암호화(SSE) 사용, 전송) ▪ High Availability / Scaling	▪ 고객 데이터 ▪ 데이터 보호(대기 시 클라이언트 측 암호화(CSE) 사용) ▪ AWS IAM(유저, 그룹, 룰, 정책)

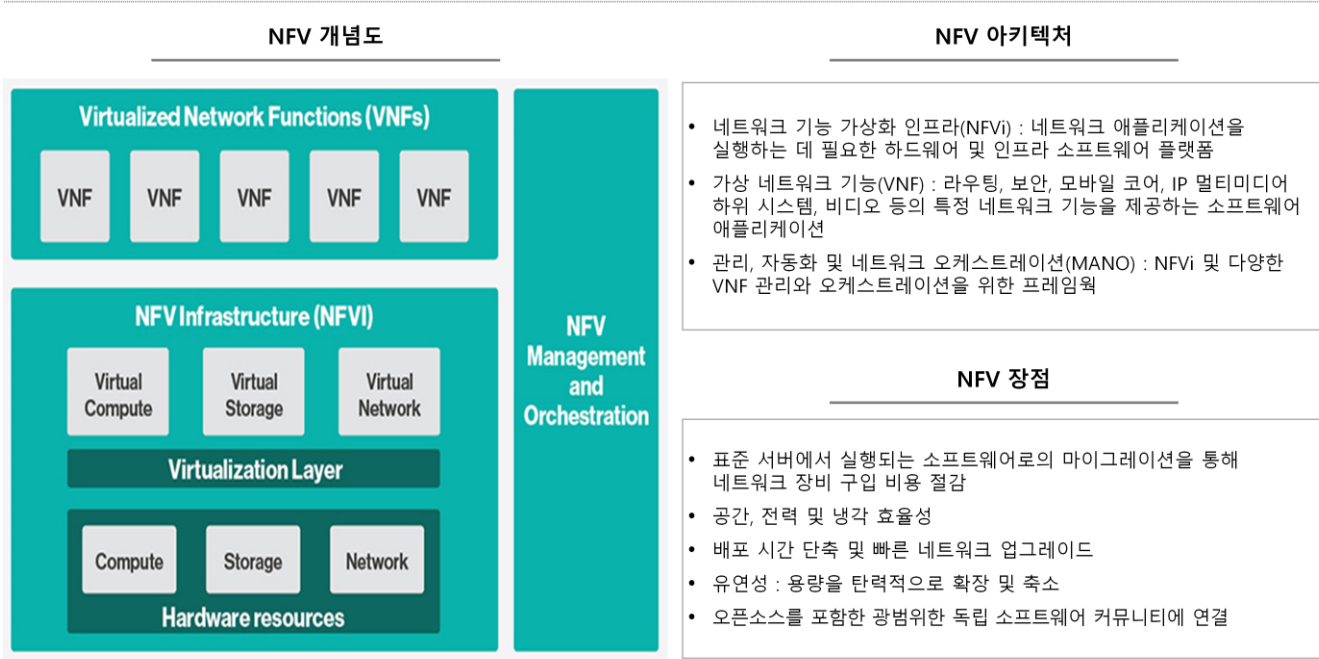
※ Check Point

- * 출처 : <https://aws.amazon.com/ko/compliance/shared-responsibility-model/>
- * **AWS의 책임**
 - 클라우드보안
 - 클라우드 환경을 구성하는 인프라에 대한 보안을 책임짐
 - 하드웨어, 소프트웨어, 네트워킹, 데이터 센터 → AWS의 서비스를 실행하기 위한 요소
- * **고객의 책임**
 - 클라우드에서의 보안
 - 클라우드에 저장하고 연결한 것에 대한 보안을 책임짐
 - 게스트 OS 관리, 고객이 인스턴스에 설치한 모든 어플리케이션 소프트웨어 혹은 유틸리티, 인스턴스별로 AWS에서 제공한 방화벽의 구성 관리 등



NFV

□ 정의: NFV(네트워크 기능 가상화)는 라우터, 방화벽 및 부하 분산 장치와 같은 고가의 전용 하드웨어 장치를 산업 표준 서버에서 가상 머신으로 실행되는 소프트웨어 기반 네트워크 어플라이언스로 대체할 수 있도록 해주는 진화한 네트워크 구성 방식임

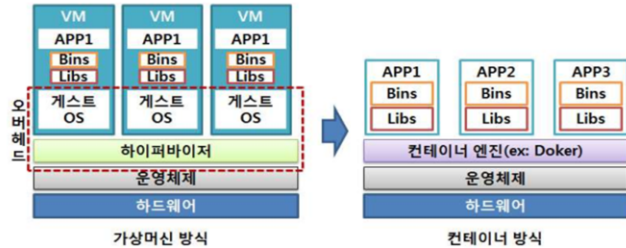


Container
(컨테이너)

- Container 정의: APP와 APP를 구동하는 환경을 격리한 공간, 컨테이너(Container)는 모듈화되고 격리된 컴퓨팅 공간 또는 컴퓨팅 환경을 의미함. 시스템 환경 의존성 탈피, 안정적 구동 등을 특징으로 함
- Docker: 응용프로그램들을 소프트웨어 컨테이너 안에 배치시키는 일을 자동화하는 오픈소스 프로젝트

컨테이너 개요

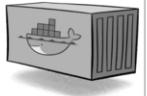
- 컨테이너(Container)는 모듈화되고 격리된 컴퓨팅 공간 또는 컴퓨팅 환경을 의미하며, 시스템 환경 의존성을 탈피하고 안정적으로 구동



※ 하이퍼바이저: 하드웨어를 가상화를 통해 하드웨어와 각각의 가상머신들을 관리하는 가상머신모니터(VMM, Virtual Machine Monitor)와 같은 중간관리자

도커(Docker)

- 특징: 리눅스 컨테이너(LXC)기술 기반
- 이식성, 이동성, 유연성 향상
- 컨테이너 이미지 생성 기능 제공으로 원격 배포하여 고도의 분산 시스템 생성 가능. 레이어 기반의 이미지 관리
- 도커 컨테이너는 소프트웨어 실행에 필요한 모든 것(코드, 런타임, 시스템 도구, 시스템 라이브러리 등 서버에 설치되는 무엇이든)을 포함하여 파일 시스템 안에 수용함



가상머신 방식과 컨테이너 기반 가상화 방식의 차이점

구분	기존방식의 가상머신	컨테이너 기반의 가상화
이식성	• VM당 모놀리딕(Monolithic)한 서비스 • VM단위의 이동, 복제와 생성 가능	• 실행에 필요한 모든 종속성 및 구성을 함께 배포 (실행환경의 일관성) • 마이크로(Micro)서비스 구축에 최적
효율성	• 1VM당 1서비스 • 성능오버헤드 존재	• 호스트 OS커널 공유이므로 필요한 만큼 자원 사용
서비스 요청에 따른 신속성	• 최소 수 GB 이상의 추가 VM을 생성하여 대응	• 게스트OS가 없는 수 MB단위의 컨테이너 생성
라이선스 비용	• VM개수만큼 지불	• Host 1대의 비용만 지불
안정성	• 각각 독립된 VM들로 안정적인 운영 가능(완전한 분리)	• 통제된 영역이지만 OS커널을 공유하므로, 장애발생시 같이 영향 받음

Hypervisor 기반 기술의 단점

- 기능 중복 이슈 (Host OS - Guest OS), 프로세스 스케줄링, 15~20% 오버헤드 발생, 수십~수백 개의 VM 배치(deployment)의 어려움

컨테이너 기반 가상화

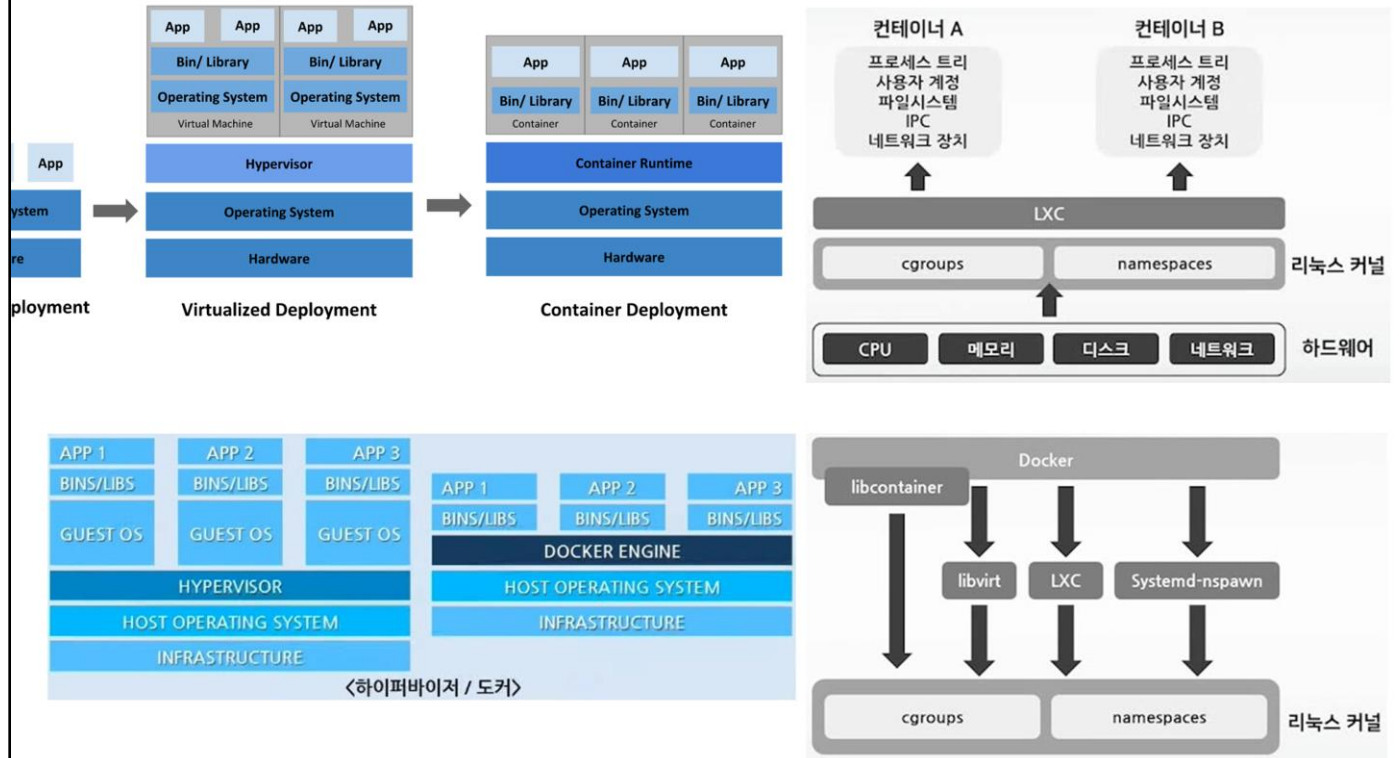
- Docker: Build - Ship(배포) - run ; 5% 이내 오버헤드 (최대 3% 이내) ; 컨테이너 간 격리 환경 구성 (※ google go 언어로 작성(DotCloud사 → Docker Inc. 변경))
- 동일 (최초 ubuntu / debian에서 RHEL/cent/fedora Linux only) OS 커널 공유
 - . 미지원 OS: Windows, Mac
 - . LINUX native 기능인 LXC 기능 이용
 - Libcontainer라는 OS 독립적인 실행 드라이버(exec driver)로써 LINUX 의존도 감소 노력 중
- ※ Container에는 Minimized OS 또는 runtime SW가 탑재되어 filesystem 등을 핸들링 함.

aufs(Advanced multi-layered Unification File System): 중복된 파일/시스템 등의 요소 공유

- e.g. Ubuntu image와 Ubuntu 기반 Java 환경 image가 있다면, Ubuntu OS은 환경 공유하고 Java 등과 같은 차이 나는 요소만 설치하여 관리하는 파일 시스템 (외부에서는 두 개 이미지)
- Git 등 활용, 컨테이너/이미지 버전컨트롤 기능 적용, 기타 부가 기능 보유
- native Ubuntu OS와 docker 기반 Ubuntu OS는 약간 상이함
 - e.g. 환경변수 설정, 서비스 실행 방식 등

LXC : Linux Container, Ubuntu 16.04 LTX 버전에서 기본 포함

- 시스템 레벨 가상화 ← chroot(change root) 명령어에서 발전
 - // cgroups (provisioning) // Namespace isolation // jail control
- LXD: Ubuntu 만든 canonical사에서 만든 컨테이너 솔루션(Container Hypervisor)
 - . 일종의 하이퍼바이저로서 KVM을 경쟁 기술로 간주함
 - . LXC + 보안 (root 생성 제약 품)
 - . Machine Container 이므로, 이 위에서 docker 수행 가능
 - . 데몬 lxd + 커맨드 기반 클라이언트 lxc + nova-compute-lxd (OpenStack Nova Plugin)
 - . scalable, 동일 HW 사양으로 KVM보다 많은 instance 구동 가능 (94% 향상, 거의 두 배)
 - 14.5배 이상의 밀도(density) 제공, 실행 시 57% 더 적은 지연(latency) 제공



Hypervisor 기반 기술의 단점

- 기능 중복 이슈 (Host OS - Guest OS), 프로세스 스케줄링, 15~20% 오버헤드 발생, 수십~수백 개의 VM 배치(deployment)의 어려움

컨테이너 기반 가상화

- Docker: Build - Ship(배포) - run ; 5% 이내 오버헤드 (최대 3% 이내) ; 컨테이너 간 격리 환경 구성 (※ google go 언어로 작성(DotCloud사 → Docker Inc. 변경))
- 동일 (최초 ubuntu / debian에서 RHEL/cent/fedora Linux only) OS 커널 공유
 - . 미지원 OS: Windows, Mac
 - . LINUX native 기능인 LXC 기능 이용
 - . Libcontainer라는 OS 독립적인 실행 드라이버(exec driver)로써 LINUX 의존도 감소 노력 중
- ※ Container에는 Minimized OS 또는 runtime SW가 탑재되어 filesystem 등을 핸들링 함.

- Aups(Advanced multi-layered Unification File System): 중복된 파일/시스템 등의 요소 공유

- e.g. Ubuntu image와 Ubuntu 기반 Java 환경 image가 있다면, Ubuntu OS은 환경 공유하고 Java 등과 같은 차이 나는 요소만 설치하여 관리하는 파일 시스템 (외부에서는 두 개 이미지)
- Git 등 활용, 컨테이너/이미지 버전컨트롤 기능 적용, 기타 부가 기능 보유
- native Ubuntu OS와 docker 기반 Ubuntu OS는 약간 상이함
 - e.g. 환경변수 설정, 서비스 실행 방식 등

LXC : Linux Container, Ubuntu 16.04 LTX 버전에서 기본 포함

- 시스템 레벨 가상화 ← chroot(change root) 명령어에서 발전
 - // cgroups (provisioning) // Namespace isolation // jail control
- LXD: Ubuntu 만든 canonical사에서 만든 컨테이너 솔루션(Container Hypervisor)
 - . 일종의 하이퍼바이저로서 KVM을 경쟁 기술로 간주함
 - . LXC + 보안 (root 생성 제약 품)
 - . Machine Container 이므로, 이 위에서 docker 수행 가능
 - . 데몬 lxd + 커맨드 기반 클라이언트 lxc + nova-compute-lxd (OpenStack Nova Plugin)
 - . scalable, 동일 HW 사양으로 KVM보다 많은 instance 구동 가능 (94% 향상, 거의 두 배)
 - . 14.5배 이상의 밀도(density) 제공, 실행 시 57% 더 적은 지연(latency) 제공

Kubernetes
(쿠버네티스)

□ 정의: Container 등을 효율적으로 관리하기 위한 Orchestrator, Master와 Node로 구성

. 엔터프라이즈급 컨테이너 배포 및 관리를 위해 구글에서 공개한 대표적인 프로젝트이며, 다수의 컨테이너의 실행을 관리 및 조율하는 컨테이너 오케스트레이션(Orchestration)이 주역할

쿠버네티스(Kubernetes) 특징

쿠버네티스(Kubernetes) 오케스트레이션 기능

• Gmail과 Google Drive 운영환경에 사용 • 다양한 테스트를 만족하는 안정적인 솔루션 • 기술자료가 매우 풍부 • Fault-tolerance • Vendor Lock In(벤더 종속성)이 이전보다 좀더 자유로워짐 • 시스템의 경량화 • 마이크로서비스의 용이함 • 컨테이너의 이동성 • 스케일링 향상 • 유연한 결합성		기 능		내 용
		서비스 디스커버리 (Service Discovery)		서비스 탐색 기능으로 기본적으로는 Cloud 환경에서 컨테이너의 생성과 배치 이동여부를 알 수 없기에 IP, Port 정보 업데이트 및 관리를 통해 서비스를 지원함
		스케일링 (Scaling)	로드밸런싱 (Load Balancing)	생성된 컨테이너의 컴퓨팅자원 사용량의 설정 및 자동배분
			스케줄링 (Scheduling)	늘어난 컨테이너를 적합한 서버에 나누어 배포하고, 서버가 다운될 경우 실행 중이던 컨테이너를 다른 서버에서 구동시킴
		클러스터링 (Clustering)		여러 개의 서버를 묶어 하나의 서버처럼 사용할 수 있도록 지원하거나, 가상네트워크를 이용하여 산재된 서버를 연결시켜줌
		로깅/모니터링 (Logging/Monitoring)		여러 개의 서버를 동시에 관리할 경우 한 곳에서 서버 상태를 모니터링 하고 로그 관리를 할 수 있도록 함

- ※ Checkpoint
- 컨테이너의 제약이 K8s의 탄생 배경임. 컨테이너화된 애플리케이션을 배포, 관리, 스케일링(축소 및 확장)하는 지겹고 복잡한 작업을 도와줄 구원 투수 SW.
 . 성장 이유: On-promise / Cloud 양 영역 모두에 앱 이식 가능
 - 원문 보기:
 . <http://www.ciokorea.com/news/126861#csidx7265f01ab895ee0b003f457f1a0043e>
 - K8s 네트워크 플러그인 : https://blog.naver.com/alice_k106/221574467441

NHN CLOUD NKS 서비스 체계

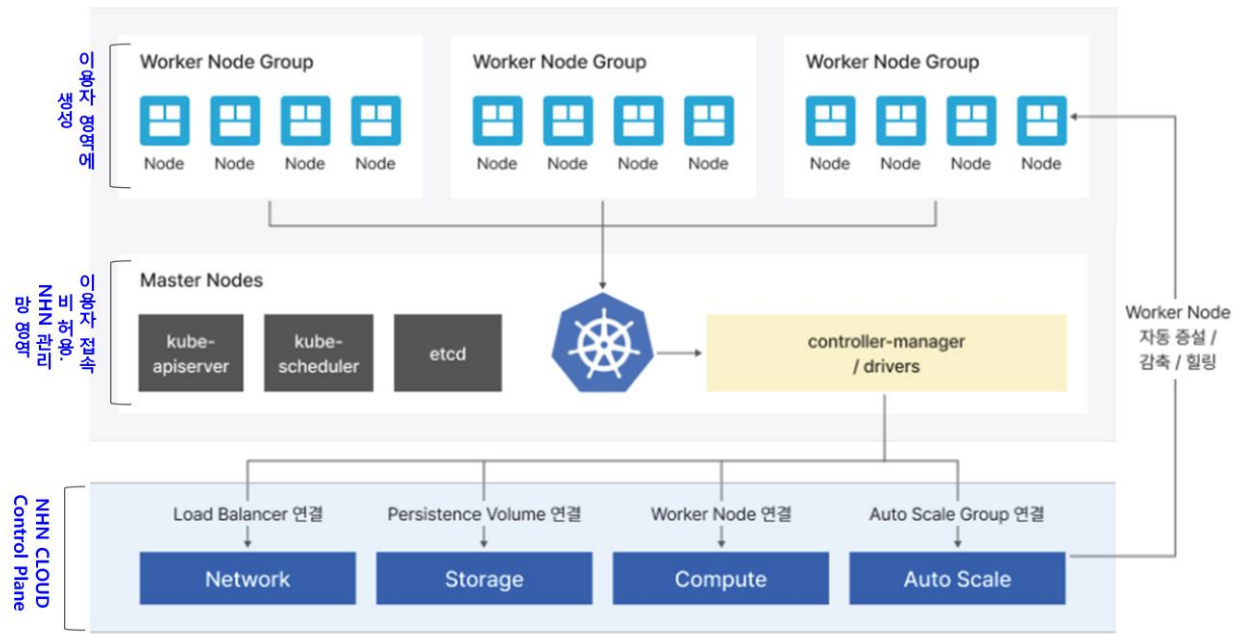


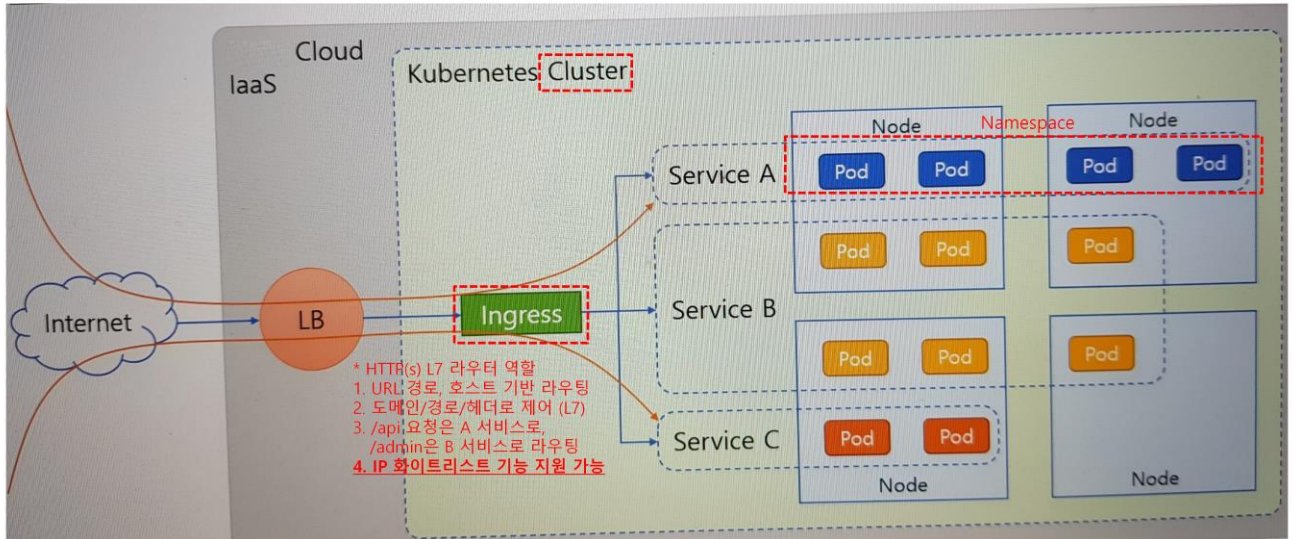
그림 7-2 NHN Kubernetes Service 특징

※ Checkpoint

Cluster_Service_Node_Container 구성 체계 샘플

※ 금융권 망분리 시 거론되는 K8s의 Cluster와 Namespace(Project), Ingress는 본문과 슬라이드 노트 참조

- Namespace는 최소 단위는 POD이며, 하나 이상의 POD가 Namespace를 공유 가능하며, 여러 노드에 걸쳐 실행 가능함.
- 제1금융권 및 유명 금융기업의 경우, Cluster와 Namespace를 적절히 동시에 적용함.



36

※ Checkpoint

- **POD 특성.** Container와 POD와의 관계 등

- . Container < POD < Node < Cluster ※ 서비스의 구성은 Node와 multi-AZ 혼합 구성 가능
- . POD: 쿠버네티스의 최소 실행 단위. Scaling, Replication의 단위, 논리적인 호스트, VM과 유사 동작. 컨테이너 그룹 (권장: 1개 container + 1 개 이하 Side Car Container)
- . 워커 노드에서 실행
- . K8s Namespace 공유 ※ Namespace 리소스를 논리적으로 구분하는 단위.
 - * 하나의 Namespace에는 여러 POD가 소속 가능. 이러한 여러 POD들은 서로 다른 노드에 분산되어 실행 가능.
 - K8s Namespace의 최소 단위가 POD라는 시각
 - * 리소스: 실행 단위인 POD, Service, deployment, ConfigMap, secret 등 포함
- . 고유의 IP 할당 단, port pool 공유를 통한 port 사용 제약 (서로 다른 pod에서는 port 중복이 가능함)
- . POD 내부 리소스 공유, 시스템 자원 공유, 단, 파일시스템은 POD 단위 공유가 아닌, Container 단위 공유
- . Pod 간 네트워크 구성은 FLAT 네트워크 구조를 따름.

* 참고 : AWS EKS / Az AKS / NHN Cloud NKS 등의 경우,

- worker node는 subnet 단위로 생성됨.
- 정책 오픈 시 구성 EC2 / VM 등에 접속 가능함. (← EKS Node image 등이 AMI / Template 으로 관리될 수 있음.)
- Cluster와 Worker Node의 범위 비교: Cluster > Node

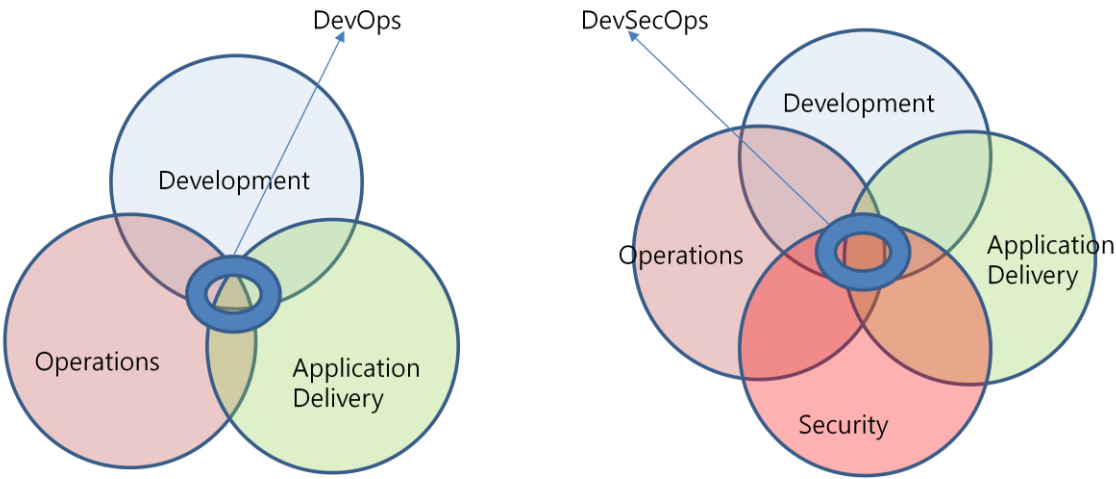
* K8s Ingress의 IP whitelist 기능 적용 예제

- Ingress는 기본적으로 ACL 기능을 포함하지 않지만, Ingress Controller의 기능이나 LoadBalancer 설정을 통해 IP 기반 접근 제어를 구현 가능함. 포트 기반 제어는 일반적으로 Service 레벨(NodePort, LoadBalancer)에서 다룸.
- YAML: `nginx.ingress.kubernetes.io/whitelist-source-range: "192.168.1.0/24"`
- AWS 환경: `loadBalancerSourceRanges`를 활용해 특정 CIDR만 접근 가능하게 설정

DevOps

- 개념: 개발과 운영의 합성어로 개발과 운영 간의 상호 작용을 원활하게 하기 위한 개발 방법론
- 목적: 개발자와 비 개발자 사이의 대화, 협동, 통합을 강조하고 담당 업무와 직급 간 상호 이해를 추구
 - . 신뢰성, 보안성 기반 빠른 개발→테스팅→배포 순환

DevOps vs DevSecOps



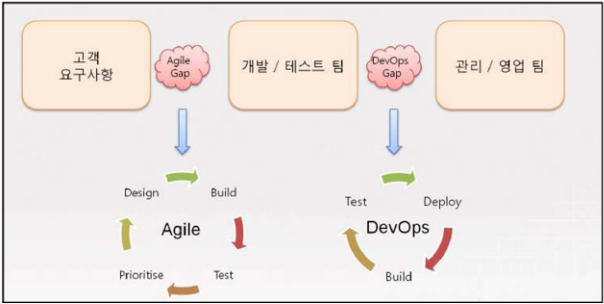
DevOps 구성요소

구분	구성요소	요소 설명
품질	• 테스트 자동화	- 서비스 통합/시스템 테스트 - 단위테스트(xUnit), UI 테스트
	• 프로세스 내재화	- 업무백로그-운영을 프로세스화 - 운영 이슈 개선 위한 프로세스
프로세스	• 소규모 릴리즈	- 변화에 유연, 롤백 용이 - Cycle-time 감소, 피드백 강화
	• 지속적 통합 (Cont. Integration)	- 개발초기-릴리즈 간 요구 반영 - CMDb 코드 통합, Test, Release
도구	• 어플리케이션 릴리즈 자동화	- 개발팀은 형상관리 기반 배치 - 운영팀은 표준화GW 사용
	• 프로비저닝	- 코드 자동 설치, 시스템 구성 - Tool: 퍼펫(Puppet), 셰프(Chef)

DevOps 한계점 및 고려사항

구분	한계점	고려사항
장애	• 버그 자주 발생 • 테스트 부족	- 표준, 자동화 프로세스
보안	• 개발자 운영 접근 • 내부 부정	- 우선순위 명확화 - 재 사용성 고려
인력	• 개발자 업무 비효율 • 평가기준 모호	- 개발, 운영 분리 운영 - 탐지통제와 감사강화

Agile 방법론과 DevOps의 개념 비교

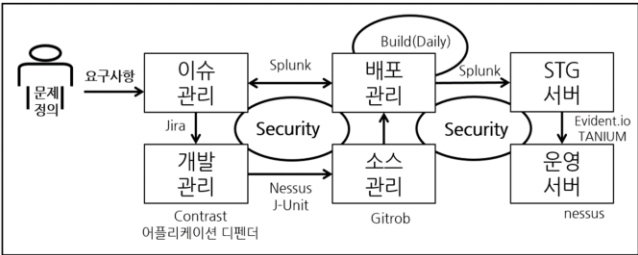


항목	Agile 방법론	DevOps
개념도		
개념	• 변화에 유연하고 신속하게 적응하는 시스템 개발 방법론	• 개발과 운영의 합성어로 개발과 운영의 상호작용 하게 하는 개발 방법론
목적	• 빠른 환경 변화 적응 • 고객 요구 적극 수용	• 개발팀과 운영팀의 원활한 Co-work
중점가치	• 개인과 상호작용 • 고객과 협력 • 변화에 대한 적응	• 개발팀과 운영팀의 원활한 의사소통 • 원활한 운영, 서비스

DevSecOps

□ 정의: 보안팀, 프로세스 및 툴을 DevOps에 통합하여 보안팀과 개발팀 간 장벽을 해소한 공동 작업 기법

DevSecOps 라이프사이클



※ 개발 전 주기에 보안을 강화하여 수명주기 확보하며 검증강화

DevSecOps 적용을 위한 고려사항

고려사항	주안점
• 반복의 이해	- 개발과정 반복, 보안, 개발 중요성 인지
• 마찰 제거	- 보안이 개발 저해 요인이 아님 인지
• 개발팀 입장	- 보안 전문가들의 개발 프로세스 이해
• Bottom-Up 방식	- 개발과 보안의 상충을 고려

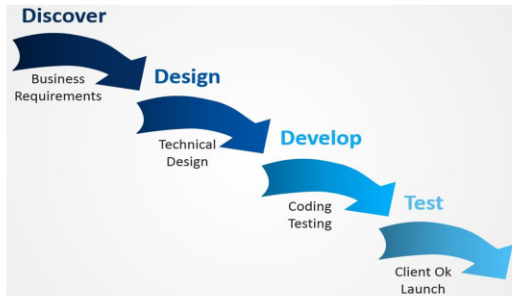
DevOps 한계점 및 고려사항

구분	기술 요소	설명
이슈 관리	• Jira	- 이슈 사항 등록, 모니터링
	• Splunk	- SIEM기반 보안 모니터링, 시각적
개발 관리	• CONTRAST	- 어플리케이션 정적 분석도구
	• J-Unit	- 테스트 코드 자동생성, 테스트수행 소스
소스 관리	• Gitrob	- Github 소스 분석 도구
	• SVN	- 버전 별 소스 관리/분석 도구
운영 환경	• Evident.io	- Cloud 보안 모니터링
	• nessus	- 보안 취약점 스캐너

Agile

- 정의: 문서작업 및 설계에 집중하던 개발 방식에서 벗어나 좀더 프로그래밍에 집중하는 개발 방법론
- Agile(애자일)이란 단어는 '날렵한', '민첩한'이란 뜻을 가진 형용사이며, 정해진 계획만 따르기보다, 개발 주기 혹은 소프트웨어 개발 환경에 따라 유연하게 대처하는 방식을 뜻함
- 모든 것을 계획 및 구축하여 작업을 진행하는 방식 대비 빠르게 변화하는 환경에 대응하며 소프트웨어를 개발

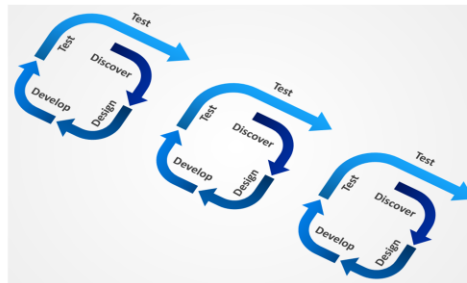
워터폴(Waterfall) 방법론



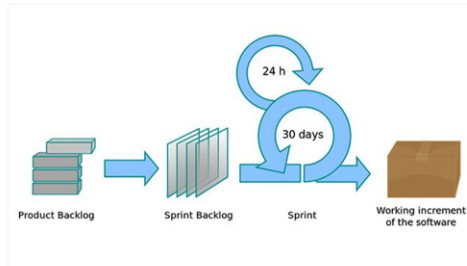
[워터폴 모델 도식화, 출처:http://slidehunter.com]

- '폭포수 모델'로도 번역되는 워터폴 방식은 소프트웨어 개발의 한 단계, 한 단계 순차적으로 수행하는 방식
.요구분석 → 설계 → 디자인 → 코딩 → 개발
- 장점: 오랜 기간 사용된 기법으로 적용 사례가 많고, 단계별로 정형화된 접근방식을 사용하여 기술적인 위험요소가 적음
- 단점: 요구사항 분석과 타당성 검토에 많은 시간과 노력이 들어가고 그 과정이 끝나면 중간에 요구사항을 추가하거나 변경하는데 어려움이 있음

애자일(Agile) 방법론



[애자일 모델 도식화, 출처:http://slidehunter.com]



[스크럼 프로세스, 출처:위키백과]

- '애자일 모델'은 전체적인 플랜을 짜고 문서를 통해 주도해 나가는 방식
- 일정한 주기를 가지고 끊임없이 프로토타입을 만들어 냄
- 필요할 때마다 요구사항을 더하고 수정하여 커다란 소프트웨어를 개발해 나가는 방식
- 애자일은 일반적으로 스크럼 프로세스를 따름
- 스크럼은 스프린트를 반복하여 제품을 언제든지 출하할 수 있는 상태를 유지한 채 완성시키는 개발방법
- 스프린트는 전체 개발 기간을 1~4주간격으로 나누어 개발을 수행하는 기간

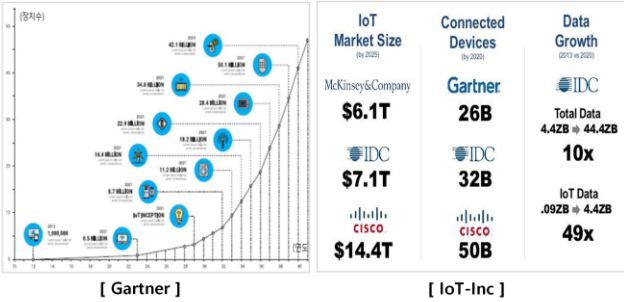
Cloud to Edge

- 정의: 다양한 단말 기기에서 발생하는 데이터를 Cloud와 같은 중앙 집중식 데이터센터로 보내지 않고 데이터가 발생한 현장 혹은 근거리에서 실시간으로 처리하는 방식(정보통신 용어사전, 한국정보통신기술협회 2018)

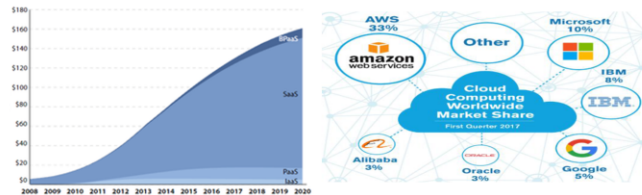
□ 즉, 사용자의 디바이스 근처, 네트워크의 가장자리에서 컴퓨팅을 지원하는 것으로 Cloud 컴퓨팅과 서비스를 네트워크 말단부로 확장한 것

엣지 컴퓨팅 확산 배경

5G 시대의 도래와 함께 IoT 기기의 폭발적인 증가에 대응하기
역부족일 것으로 예상되는 Cloud 컴퓨팅 기술에 대한 대안으로
엣지 컴퓨팅 기술이 빠르게 확산



- Gartner는 2017년 기준 전세계적으로 보급된 IoT 장치수를 약 84억개로 추정하면서 2020년 경에는 260억개를 넘어설 것으로 전망
- McKinsey, IDC, CISOC 등은 IoT 시장규모도 2025년경 적게는 6.1조달러에서 많게는 14.4조달러에 이를 것으로 전망
- 이와 같이 폭발적으로 증가하는 네트워크 트래픽에 대응하는 기술로 최근까지 Cloud 컴퓨팅이 빠르게 확산



- 최근 들어 5G 시대가 본격적으로 도래하여 데이터 트래픽이 폭발적으로 증가할 경우 Cloud 컴퓨팅 기술로 대응하기에는 역부족이라는 의견이 지배적

안정성	자율주행 자동차	순간의 네트워크 지연이나 데이터 전송 오류가 치명적인 사고로 이어질 수 있음
	항공엔진, 드론	
즉시성	연안 석유시추시설	산업기계 자체가 중앙 서버에서 멀리 떨어진 곳에 위치해 있어 중앙서버와의 연결이 어려움
	증강현실	사람의 시청각 반응 능력은 매우 예민하기 때문에 불과 몇 백 ms 차이만으로도 가상현실 몰입감에 영향을 미칠 수 있음
	가상현실	
효율성	생체인식	
	스마트 팩토리	제조 기업의 스마트 팩토리에서는 대규모의 센서 데이터가 발생하며 이의 효율적인 처리가 필요함

[Cloud 컴퓨팅 기술 한계 예시 출처:삼성뉴스룸, 정보통신정책동향]

엣지 컴퓨팅 개념

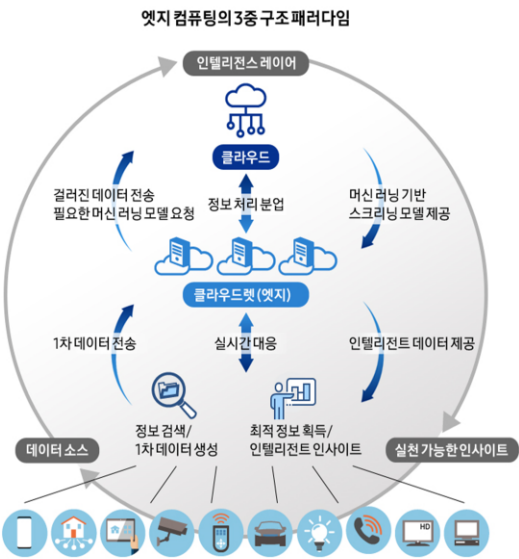
- 최근 인터넷 기업들이 주도하는 Cloud 컴퓨팅의 한계를 극복할 수 있는 대체기술로 엣지 컴퓨팅(edge computing)이 주목
- 엣지 컴퓨팅은 Cloud 컴퓨팅과 정반대 방식의 기술로 Cloud 컴퓨팅이 사용자 기기의 통제가 중앙 데이터센터에서 주로 이뤄지는 방식이라면, 엣지 컴퓨팅은 사용자 기기(혹은 로컬 영역의 지점)에서 직접 컴퓨팅이 이뤄지는 기술

엣지 컴퓨팅 특징

안정성	네트워크 의존도를 낮추고 안정적으로 데이터 송수신 가능 - 데이터가 기기에 직접 연결 및 접속 가능한 네트워크 노드 상에 위치하고 있어 지연이나 전송 오류 낮음 - 데이터 해킹과 같은 공격으로부터의 노출 적음
즉시성	Cloud 컴퓨팅보다 빠른 연산과 응답이 가능 - 기기와 데이터의 물리적 거리가 짧고 연산을 위해 다루는 데이터의 양도 Cloud 컴퓨팅에 비해 적기 때문에 더 빠른 연산과 데이터 전송이 가능
효율성	기기에서 발생되는 데이터 대부분을 자체적으로 처리 - 기기에서 수집되는 막대한 데이터의 대부분을 엣지 컴퓨팅으로 처리하고 중앙 데이터센터에는 꼭 필요한 데이터만 전송 - 이를 통해 데이터 부하를 줄이고 전송 단계에서 보안 위협도 피함

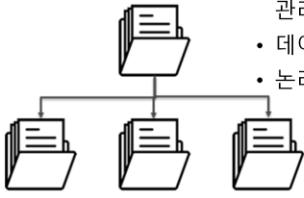
엣지와 Cloud 사이의 관계

- 엣지 컴퓨팅을 통해 기존 Cloud 컴퓨팅이 가진 한계를 극복 또는 보완



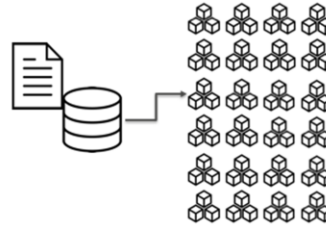
[엣지와 Cloud 사이의 관계, 출처:삼성뉴스룸(2017)]

File Storage



- 범용성 우수
(이용자가 직접 파일을 편하게 관리하는 것에 주안점을 두어 개발)
- 데이터는 폴더 내 단일 파일 저장
- 논리적 계층(tree) 구조
- 파일 권한 관리 및 SW를 통한 접근 용이
- NAS (NTFS, FAT 유형, POSIX File System 지원 등)

Block Storage



- 데이터를 Block으로 분리. 분리된 Block은 분산 저장 가능. Block은 독립적으로 존재. (파티션 분할 가능)
- disk를 Block으로 독립된 (서버, 서로 다른 OS) 시스템에 제공
- 범용 OS 연계, 통제 지원
- 다양한 데이터 경로 지원 통한 신속 검색 지원
- SAN, VMware's VMFS 등

Object Storage



- RAID를 대체하는 차세대 데이터 스토리지 아키텍처
- 동일 데이터 저장 시, 파일 스토리지는 3배 정도 용량 소모, OBS는 1.87배 용량 소모

- 저장 데이터 : object(Binary Data) ※ 수정 불가능. 덮어쓰기 개념으로 동작 (N사 Cloud의 경우, 선택적으로 '객체 잠금 기능' 제공)
- 저장에 필요한 정보들 : **고유 식별 ID**, 메타데이터(사용 기간, 날짜 정보 등, 정보가 다양할수록 검색에 유리), object
- 저장 방식 : 데이터를 여러 object로 (다수 디스크 or 시스템에) 분산 저장. **Chunk('청크') 단위 저장**, 타 IDC에도 저장 가능. **리전간 컨테이너 복제 지원**.
- 저장 시 기술적 전제 : object 저장을 위해 OBS 컨테이너 필요 (예: AWS S3 Bucket -object 저장 용도 대용량 컨테이너-)
- 데이터 접근 : push, get ※ HTTP API 필요 (push, get, delete 등)
상기와 같은 **폐쇄적 특징**으로 DBMS 직접 연결 등이 파일 스토리지 대비 난해함. CSP가 제공하는 API 등으로 추가 개발 필요
- Write Once, Read Many 스타일의 **(비정형)** 데이터 저장 유형에 유리, 비 구조화된 **대용량** 데이터 저장에 유리

※ Checkpoint

- 버킷 : 버킷은 데이터를 담는 기본 컨테이너입니다. Cloud Storage에 저장하는 모든 컨테이너가 버킷에 포함되어야 합니다. 버킷을 사용하여 데이터를 구성하고 데이터 액세스를 제어할 수 있지만 디렉터리와 폴더와 달리 버킷을 중첩할 수 없습니다.
- * 유사 : GCP, NHN Cloud – Storage Container

SECaaS

(Security as a Service)

- 정의: SaaS의 한 종류로 보안서비스를 인터넷을 통해 온디맨드 형태로 제공하는 서비스 모델
- On-premise 형태의 보안 인프라에 비해 비용이 저렴하고 관리 포인트가 줄어 부담이 적음
- 기존솔루션을 SECaaS로 대체하고 관제서비스와 결합해 서비스를 제공하는 형태로 변화

서비스 종류

Network Security

Web Security

신원 및 접근 관리

침입 관리

보안 정보 및 이벤트 관리(SIEM)

지속적 모니터링

취약점 스캐닝

이메일 보안

암호화

데이터 손실 방지

사업 연속성 및 재해복구

보안성 평가

장점

• 솔루션 설치 비용 등 별도의 초기 비용 없음

• 업데이트 및 유지관리 비용이 없음

• 인터넷 및 전용선 대역폭 절감이 가능

• 내부 보안인력에 투입되는 비용 절감이 가능

국내 주요 사업자 현황

업체 명	대표서비스	주요기능
모니터랩	Cloud WAF	Cloud 기반 웹 방화벽
안랩	사이트 스키퍼	웹사이트 악성코드 위변조 탐지
이스트 시큐리티	서비스명 미정	Cloud 환경에 최적화된 백신 서비스 예정
지란지교시큐리티	Jiran the CLOUD	Cloud 기반 Email 보안 서비스
Cloud브릭	Cloud브릭	Cloud 기반 웹방화벽

CASB
(Cloud Access
Security Broker)

- 정의: 2013년 가트너가 정의한 기술로, Cloud 이용에 있어 안전을 담보해주는 중개자 기능을 제공하는 새로운 보안 레이어를 의미
- Cloud 사업자와 사용자 중간에 위치해 보안 전반을 책임지는 Cloud 보안 중계 서비스

기능

유형 별 특징

- Cloud 서비스 검색 및 위험평가
- 암호화
- 접근 통제
- 데이터 손실 방지 (DLP)
- 로깅 및 감사
- 이상탐지
- 각 서비스의 위험 평가
 - 회사에서 사용 중인 모든 Cloud 서비스 검색
 - 기업 제어 암호화 키를 사용하여 Cloud 데이터 암호화
 - 사용자, 장치 또는 위치의 컨텍스트를 기반으로 Cloud 서비스에 대한 접근 제어
 - 사용 중인 DLP 솔루션과의 통합을 통해 Cloud로 이동하는 데이터 DLP 적용
 - Cloud에서 수행한 작업의 로깅 및 감사 추적을 통해 보안정책 준수 여부를 검증하고 위반 행위 조사
 - 잠재적인 데이터 유출, 보안 위반 등을 판단하여 보안 관리자에게 경고

구분	에이전트형 CASB	프라이빗 CASB	퍼블릭 CASB	API형 CASB
형상	소프트웨어	하드웨어	하드웨어	소프트웨어
설치 위치	이용자 단말 내	이용자 조직 내	Cloud 서비스 플랫폼 내	Cloud 서비스 플랫폼 내
이용 목적 주체	Cloud 서비스 이용자	Cloud 서비스 이용 기업	Cloud 서비스 제공자	Cloud 서비스 앱 개발사
주요 기능	접근 통제	접근 통제, DLP	접근 통제	접근 통제
SSL 처리 여부	불필요	처리	불필요	불필요
특징	이용자 OS에 따라 별도 개발	에이전트 설치 필요	Cloud 서비스 규모에 따라 과부하 발생하지 않도록 설치 필요	Cloud 서비스 어플리케이션 환경 별로 개발 필요

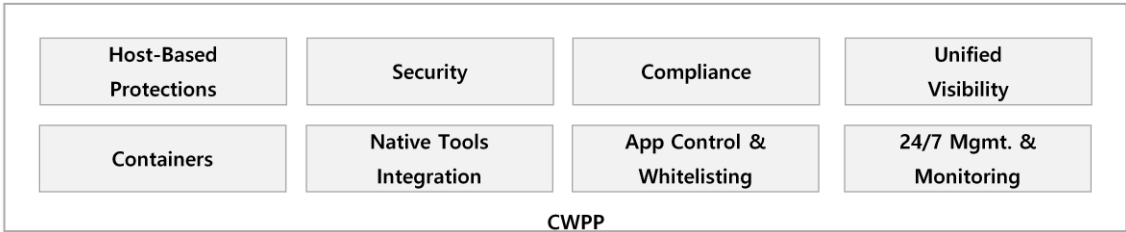
- CWPP
(Cloud Workload Protection Platform)

□ 정의: Cloud 서버 워크로드 중심의 보안 방어(protection)을 위해 설계된 제품

□ Agent Based 형식으로 시스템 강화, 취약점 관리, 호스트 기반 분할, 시스템 무결성 등의 기능 제공

□ 단일 콘솔에서 Multi Public Cloud 환경 가시성 확보 및 보안 제어

CWPP 주요 기능



- 프로비저닝 통합 보안 기능을 호스트 레벨에서 제공하여 Cloud 워크로드 보호
- 퍼블릭 Cloud, 프라이빗 Cloud, 컨테이너 및 가상화 환경을 위한 Cloud 기반 워크로드에 대한 통합 가시성 및 제어 기능 제공
- 가능한 네이티브 서비스와 통합
- 주요 규정 준수 제어 문제 해결
- 조직이 Shared Responsibility Model의 역할을 안전하게 보호하도록 지원
- IT, IT 보안 및 DevOps 요구사항을 해결하는 Cloud 솔루션 제공(실시간)

CWPP 주요 벤더 및 제품

- AWS (Inspector)
- Microsoft (Azure Security Center)
- McAfee (Cloud Workload Security)
- Symantec (Cloud Workload Protection)

CSPM
(Cloud Security Posture Management)

□ 정의: Cloud 서비스 구성 위험 평가 및 보안 현황 관리 솔루션

□ IAM 서비스, 네트워크 연결/구성, 스토리지 구성 및 PaaS 서비스 등 관리

□ Cloud 스택 전반의 보안 평가 및 컴플라이언스 모니터링 수행

CSPM 주요 기능



- 단일 콘솔에서 Multi Public Cloud 리스크 가시성 확보
- 사용자 지정 보안 정책 제공
- 업계 표준 및 규정에 대해 즉시 적용 가능한 정책을 통해 컴플라이언스 여부를 평가 및 리포트
- 고도화 지능 공격을 위한 동적 Rule 정의
- 우선순위에 따른 자동 조정 조치
- Cloud 자산 중요도에 따른 Alert

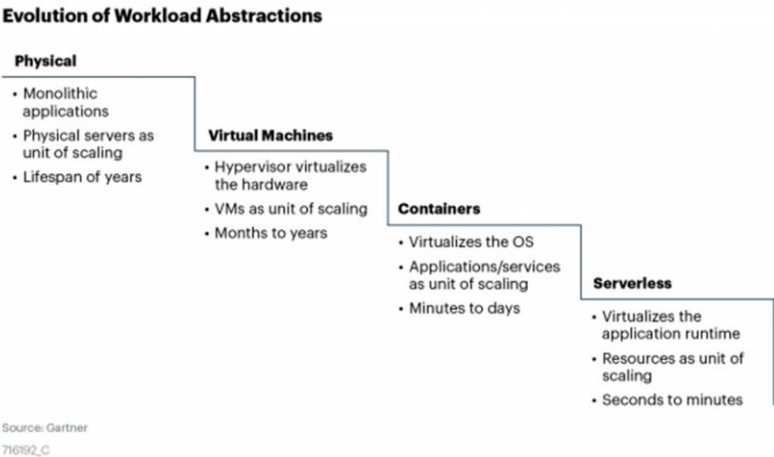
CSPM 주요 벤더 및 제품

- Turbot (TURBOT)
- Symantec (Cloud Workload Assurance)
- Check Point (Dome9)

workload

□ 정의: 단위 시간 내 수행해야 하는 업무의 양과 질을 -'load'/'부하'로 통칭- 의미함
⇒ 독립적으로 운영 가능한 업무의 최소 단위 (e.g. 여신 중의 채권관리는 독립 수행 가능)
※ payload: 총 네트워크 트래픽 양에서 통신을 위해 소모되는 트래픽을 제외한, 실제 콘텐츠 트래픽만을 계산하는 '과금 부하'를 의미함

- '워크로드'는 클라우드로 Appl.을 이전하거나 신규 구축하기 위한 최소한의 업무 단위로 인식하면 됨.



▲워크로드 추상화의 진화 단계 (자료: 가트너, 데이터넷(<http://www.datanet.co.kr>))

IaC

(Infra. as Code)

- 정의: IT Infrastructure를 Code로써 관리하는 방식을 의미함. 주로 Cloud Infra.를 관리하는데 사용됨.
- AWS cloudFormation, HashiCorp's Terraform 등이 유명함.
- 기존의 Infra. 운영 방식 대비 IaC의 가장 중요한 이점은 하드웨어 프로비저닝과 배포. 그리고 유지보수에도 적용되는 빠른 속도와 민첩성임.

□ 일부 초기 구성 관리 제품은 IaC를 포함하도록 업데이트됐다. 퍼펫, 셰프(Chef), 솔트스택(SaltStack), CFengine 이 여기에 포함된다. 모두 퍼블릭 클라우드 플랫폼에서 자동화된 코드 프로비저닝을 관리할 수 있게 해준다. 그러나 IaC는 그보다 한 걸음 더 나간다. 업체마다 동일한 구성요소에 대해 다른 명칭을 사용하기 때문에 다소 혼란스러울 수 있다. 예를 들어 퍼펫은 IaC 스크립트를 매니페스트라고 하고, 셰프에서는 레시피와 쿡북이라고 한다. 그러나 이름은 달라도 기능은 똑같이 코드 명령을 저장하는 것이다.

※ 약간 성격이 다른 또 다른 자동화 툴인 Ansible은 playbook이라 한다. (Infra. 운영 자동화에 좀 더 포커싱)

□ 각 툴에는 자체 도메인 특화 언어(Domain Specific Language, DSL)가 있지만, DSL은 일반적으로 YAML이나 JSON을 기반으로 한다. 예를 들어 테라폼은 해시코프 구성 언어(Hashicorp Configuration Language, HCL)라는 JSON 기반의 자체 DSL을 사용한다. AWS 클라우드포메이션 템플릿은 YAML과 JSON으로 만들 수 있다. 셰프와 퍼펫은 루비 언어를 사용한다

출처: V3_HPE_TI_EZMERAL_20210405

End of Documents